Green Energy and Intelligent Transportation 5 (2026) 100334

Contents lists available at ScienceDirect

Green Energy and Intelligent Transportation

journal homepage: www.journals.elsevier.com/green-energy-and-intelligent-transportation

Review article

Deep learning-driven CNN model for detection and classification of dynamic obstacles

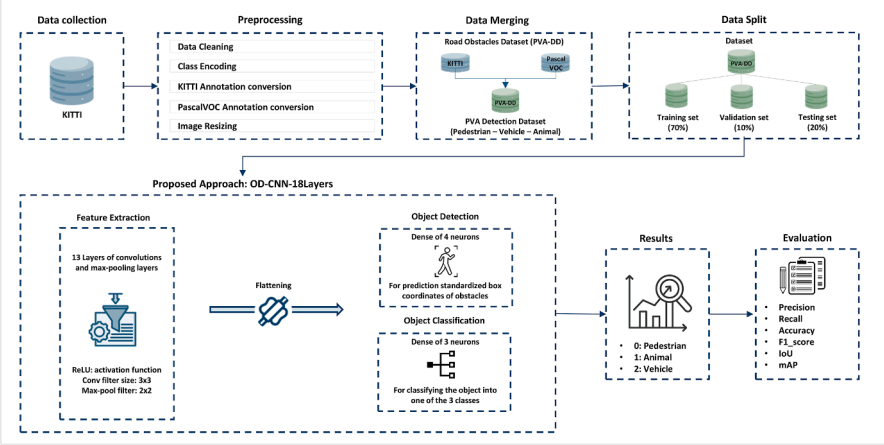
Hamza Assemlali*, Soukaina Bouhsissin, Nawal Sael

Information Technology and Modeling Laboratory, Faculty of Sciences Ben M'Sik, Casablanca 20670, Morocco

HIGHLIGHTS

- **Creation of a Comprehensive Dataset**: Developed a robust dataset that includes a diverse range of mobile obstacles such as pedestrians, vehicles (bicycles, buses, trucks, cars, motorbikes), and animals, enhancing training and evaluation for detection algorithms.
- **Proposition of an Advanced Detection Model**: Introduced a novel CNN detection model (OD–CNN–18 Layers) that surpasses existing state-of-the-art methods, demonstrating strong performance in identifying mobile obstacles. The model emphasizes detecting critical risk factors on the road, including pedestrians, animals, and vehicles.
- **Development of a Classification Model**: Designed and implemented a CNN-based classification model (OD–CNN–18 Layers) capable of accurately categorizing detected obstacles, contributing to enhanced situational awareness in dynamic environments.

GRAPHICAL ABSTRACT



ABSTRACT

In recent years, advancements in autonomous driving technologies have shown significant potential to improve road safety, although their current impact remains a subject of ongoing research and debate. At the core of this improvement is obstacle detection, a crucial element for ensuring the effective functioning and security of these systems. Despite substantial progress, challenges remain in the detection and classification of dynamic road obstacles, which pose greater risks than fixed ones; both drivers and the obstacles in question can suffer fatal consequences because of collisions with dynamic obstacles. This highlights the urgent need for systems that can alert drivers in advance, thereby preventing accidents. The current paper presents a deep learning-driven approach to detect and classify dynamic obstacles, including pedestrians, vehicles, and animals. By integrating data from various datasets, we developed and adapted CNN architecture for this task, enhancing the reliability and safety of autonomous driving systems. This approach demonstrated significant improvements under various conditions. We have gotten a classification accuracy of 99.5% and a detection precision of 97.1%. The results indicate that our architecture offers a better ability to identify and classify road obstacles, thus contributing to the advancement of autonomous driving technologies.

* Corresponding author.
E-mail addresses: assemlalihamza@gmail.com (H. Assemlali), bouhsissin.soukaina@gmail.com (S. Bouhsissin), nawal.sael@univh2c.ma (N. Sael).

https://doi.org/10.1016/j.geits.2025.100334
Received 11 November 2024; Received in revised form 9 March 2025; Accepted 27 April 2025
Available online 1 July 2025
2773-1537/© 2025 The Author(s). Published by Elsevier Ltd on behalf of Beijing Institute of Technology Press Co., Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

1. Introduction

Intelligent Transport Systems (ITS) [1] encompass a range of technological applications used to improve the efficiency and safety of transport networks. They integrate information and communication technologies (ICT) [2] to manage and control transport infrastructure more intelligently and responsively. ITS includes a range of solutions from traffic sensors and management systems to connected and autonomous transportation. The main aim of ITS is to optimize the management of the transport network in real-time, reduce congestion, improve road safety, and minimize the environmental impact of travel. It also aims to provide real-time and accurate information to road users, allowing for better route planning and reduced travel times.

Nowadays, vehicles are equipped with advanced systems to assist the driver behind the wheel to drive the vehicle with a high degree of safety through the road. These systems fall under the Advanced driver assistance systems (ADAS) [3] field, ADAS's main goal is warning the driver whenever a danger is detected, or an important action needs to be taken to avoid an accident hence guaranteeing driver's and road safety. Vehicles equipped with ADAS can detect and sense their surrounding environment, process all the captured information by its equipped sensors, and provide the correct action to the driver rapidly and accurately.

With the rapid advancement of ITS [1] in general and autonomous driving [4] technologies, in particular, road safety is changing significantly. At the heart of this development, we find real-time obstacle detection which is a key element in ensuring these modern systems' smooth operation and safety, for both the driver and the vehicle during the voyage to the required destination. Nowadays, vehicles are equipped with advanced systems to assist the driver behind the wheel to drive the vehicle with a high degree of safety through the road. Meanwhile, the autonomous vehicle must have the capability to recognize obstacles and navigate safely through the nearby objects, numerous systems were introduced in the field of obstacle detection utilizing different tools and technologies, such as LiDar, Camera, and Radar. These technologies generally handle two types of obstacles on the road, which are fixed obstacles (e.g., potholes, speed bumps …) and dynamic obstacles (e.g., vehicles, pedestrians …), these obstacles may include imperfections on the pavement and may compromise the fluidity and safety of traffic, despite these advances, detecting road obstacles remains a significant challenge, whether fixed or mobile.

Dynamic obstacles, such as pedestrians [5], animals and other vehicles, pose increased risks of accidents requiring early detection to avoid collisions. Unlike fixed obstacles (such as road bumps, traffic signs, and potholes) dynamic obstacles can change speed and position rapidly making them unpredictable. The danger of facing a dynamic obstacle is particularly acute, as the situation's dynamic can lead to severe consequences for both the obstacle and the driver. For the drivers, the ability to not react quickly can cause loss of control and in some situations, it can be mortal. For pedestrians, animals, and other vehicles the effect might be catastrophic and might lead to serious injuries or death.

Despite the enormous contributions that have been made to autonomous vehicles globally and dynamic obstacle detection specifically with the employment of recent AI technologies, limitations are still there especially since there is approximately a lack of works that combine multiple objects into one dataset, as per our research in recent work we found mainly that contributors mainly focus to two objects maximum either pedestrians or vehicles or animals, These challenges underscore the need for technological solutions that provide real-time detection and early warning systems, also that effective detection of dynamic obstacles is necessary for autonomous entities safety and their successful integration into complex road environments. To achieve this, advanced technologies such as cameras, Internet of Things [6] devices and deep learning algorithms are essential. By incorporating these methods, we can significantly reduce the occurrence of collisions involving dynamic road obstacles, thereby enhancing overall road safety.

Our paper's contributions are as follows:

(1) Merging two Comprehensive Datasets (KITTI [7] & Pascal-VOC2012 [8]): robust dataset encompassing a diverse range of mobile obstacles including pedestrians, vehicles (bicycles, buses, trucks, cars, Motorbikes …), and animals, facilitating enhanced training and evaluation for detection algorithms.
(2) Proposition of an Advanced Detection Model: Introduce a novel CNN detection model (OD–CNN–18 Layers) that surpasses the existing state-of-the-art approaches with solid performance in identifying mobile obstacles. This architecture focuses on detecting mobile obstacles, such as pedestrians, animals, and vehicles, which are also critical risk factors on the road.
(3) Development of a Classification Model: Design and implement a CNN-based classification model (OD–CNN–18 Layers) capable of accurately categorizing identified obstacles, contributing to improved situational awareness in dynamic environments.

Our approach is to optimize the efficacity and speed of existing detection & classification systems, allowing a more effective response to potential hazards. By strengthening the ability of systems to anticipate and report on these obstacles, we aim to reduce the risk of accidents while improving road infrastructure safety and efficiency.

This paper follows this structure: Section 2 examines state-of-the-art in obstacle detection & classification. Section 3 outlines our proposed methodology, covering data collection, preprocessing, and the results of our architecture. Section 4 focuses on the experiments and their outcomes, while Section 5 discusses our findings. Finally, we close our paper with Sec. 6 which summarizes our insights and discusses limitations and further steps for our research.

2. State of the art

2.1. Related work analysis

This Section presents a comprehensive overview of research and technological advances related to the detection of mobile road obstacles, including pedestrians, animals, and vehicles, with a focus on artificial intelligence methods used to ensure safer autonomous driving. This section thus allows us to situate our research in the context of existing work, while identifying the challenges and opportunities that guide our study. The study in Ref. [9] Showcases an advanced methodology for pedestrian detection by the combination of a Histogram of Oriented Gradients (HOG) for describing the gradients within an image and Local Binary Patterns (LBP) that capture texture, these complementary features serve as input for the XGBoost algorithm. Using the INRIA dataset for evaluation, the GA-XGBoost approach shows significant improvements with an F1 score equal to 0.97, demonstrating better generalization capability on external data sets such as MOT_2DMOT2015. However, the study highlights limitations such as slow extraction of characteristics, the need to reduce detection errors, and overcoming challenges related to a local minimum in the genetic algorithm. In addition, in Ref. [10] the authors propose an advanced method for pedestrian detection, combining the Faster RCNN network with skip-pool, to improve detection in complex environments where objects of interest are often small. The method improves the Regions Proposal Network (RPN) of Faster RCNN through a multilayer approach with skip pooling. This technique combines pooling results from lower layers, enhancing efficiency and accuracy. The proposed model SP-CNN is evaluated on the KITTI and Caltech datasets. On the Caltech dataset, SP-CNN achieves a detection error rate of 9.1%, surpassing concurrent methods such as MS-CNN and RPN + BF. On KITTI, it shows an average accuracy of 76.3% in moderate mode. On the other hand, the researchers in the study [11] compared popular object detectors (SSD variants, Faster RCNN, and RFCN) on the KITTI dataset, focusing on accuracy, inference time, and resource usage. The fastest is SSD MobileNetV2(14.29 ms/frame) but less accurate for

small objects, while Faster RCNN ResNet 50 offers a good balance (58% accuracy at 116 ms/frame). Faster RCNN Inception_ResNetV2 is the most accurate but the slowest (300 ms/frame). Overall, SSD models are faster and lighter, making them suitable for mobile applications, while Faster R–CNN is better for tasks requiring high precision, highlighting the importance of model selection based on industrial application needs. In the article [12], the authors propose an approach to pedestrian detection, combining information from color stereo-vision cameras with data from a thermal camera to improve detection performance. A new dataset, consisting of 4330 images from 58 video sequences recorded in urban areas, was created for this study, with data from different cameras aligned via the trifocal tensor. The methodology is based on a reconfigurable detection framework where the extraction of characteristics is separated from the classifications. The characteristics are extracted from the first layers of a pre-trained convolutional network and then classified with an AdaBoost classifier. The proposed multi-spectral approach has shown superiority over traditional methods such as HOG, with a reduction in false alarm rate and better pedestrian detection, obtaining a log-miss rate average of 9%. The same applies to [13] in which the focus was on enhancing pedestrian detection in adverse weather conditions by evaluating deep neural networks (DNN) with a new thermal dataset called ZUT, which consists of 16-bit thermal images captured under severe weather, including rain, fog, and snow. The YOLOv3 network was modified to process these images, incorporating normalization and data enhancement techniques, achieving an average accuracy (mAP) of 89.1% in harsh conditions, compared to 79.6% with 8-bit images. In contrast, Tiny YOLOv3 performed lower at 71.5%, though increased resolution and filtering improved its results. While the modifications to YOLOv3 show promise for robust sensing in harsh environments, challenges remain, such as incomplete weather coverage in the dataset and difficulties in detecting certain classes like body parts and strollers, with performance varying based on specific testing conditions and dataset annotations. In another study [14] they assess the yield of various object detection algorithms, as well as SSDInceptionV2, SSDMobilenetV1, SSDMobilenetV2, multipled, and Pednet, in the vehicle and pedestrian recognition on rural roads. The models were implemented on the NVIDIA Jetson Nano embedded device, high-performance and low-cost hardware. The methodology uses datasets of rural road images under various environmental conditions. The results show that Pednet is the most accurate for pedestrian detection with an average accuracy of 78.71%, while SSD-mobile net-v1 stands out for vehicle detection with 70.08% accuracy. The SSD-mobile net-v2, SSDMobilenetV1, and ssd-inception-v2 models are also faster and beneficial for real-time applications. However, limitations exist such as increased processing time in complex environmental conditions and overheating of the Jetson Nano during extended processing. In addition, in the paper [15] authors analyzed the performances of different CNN models for object detection, such as Faster RCNN, SSD, and YOLOv3, for the detection of pedestrians on roads. The authors made use of the COCO dataset as a subset for training and the Daimler Mono Pedestrian for testing. The applied preprocessing consists of extracting image characteristics via CNN networks, followed by the classification and prediction of enclosing boxes. The results highlight that YOLOv3 is the most efficient model for pedestrian detection in terms of accuracy and prediction speed, surpassing SSD and Faster RCNN. However, limitations remain, including the need to improve detection in difficult conditions (bad weather, night vision) and explore the impact of loss functions on models of Pedestrian detection. The study [16] in which researchers propose a pedestrian detection system designed to work effectively in adverse weather conditions, including rain. The methodology is based on the integration of a conditional generative antagonist network (Pix2Pix GAN) to transform rainy images into dry images, combined with the YOLOv3 real-time object detector. The data set used is VOC2014, containing images with pedestrian classes, and the rainy images are artificially generated for learning. Based on the results the suggested architecture optimizes pedestrian detection in rainy conditions compared to traditional methods, with an average accuracy (AP) of 46.10% for dry images versus 26.5% for unprocessed rainy images. However, the system has performance limitations, due to software and hardware limitations. On the other hand, the authors of the paper [17] presented an innovative approach for detecting large animals in road scenes, using deep learning techniques and developing a specialized dataset. Noting that the existing datasets lack images relevant to this task, the authors combined COCO and Google Open Images datasets to create a 20 000-image dataset, covering 10 animal classes such as Bear, Fox, Dog, and Elephant. Various architectures, including Faster R–CNN R-50-FPN, RetinaNet R-50-FPN, YOLOv3, and Cascade R–CNN R-50-FPN, were tested to evaluate the performance of the models. YOLOv3 was distinguished by a 0.78 mAP for all classes and for detecting a single class 0.92, with a speed of over 35 fps, while RetinaNet R-50-FPN, although faster at 44 fps, showed a 13% lower mAP. Research in the article [18] introduces YOLOV4-5D, an object detection algorithm aimed at enhancing accuracy and speed for autonomous driving by integrating deforming convolutions and redesigning sensing modules to better handle small objects. Tested on the KITTI and BDD datasets, YOLOV4-5D utilizes K-means for anchor design, resulting in improved clustering of bounding box sizes and reaching a higher mAP of 0.7013 on BDD, compared to 0.6590 for YOLOv4. It excels in detecting small objects like traffic lights and road signs while maintaining real-time performance, outperforming algorithms such as MS-CNN and SSD. The study also explores model pruning techniques, reducing size and computation time while enhancing accuracy, particularly with local pruning. Visual results demonstrate YOLOV4-5D's superiority over YOLOv4 in detecting complex objects in dense environments, establishing it as a strong candidate for autonomous driving applications. In addition, another study on developing a barrier detection & classification system [19] for self-driving vehicles. To achieve this goal, the system uses the Caffe framework to implement Faster R–CNN, based on a pre-trained ZF Net model with ImageNet. Convolutional neural networks (CNN) automatically extract image characteristics to recognize obstacles, while Faster R–CNN provides regions of interest for classifying and locating objects. Training and test data from several sets, including KITTI, iRoads, Pascal VOC 2012, and Indian roads images. The system shows high accuracy in detecting and classifying obstacles with a processing speed of more than 10 frames in each second, and a mAP up to 71.7% for KITTI. However, improvements are needed to better adapt the system to Indian road conditions and optimize performance with larger network models. In addition, the contributors in the article [20] focus on improving obstacle detection & recognition in driving environments using CNN. Good recognition of objects such as cars, people, motorcycles, buses, and bicycles is important for road safety. The methodology includes the extraction of obstacle areas from videos, their separation into sub-images, and the annotation of regions of interest (ROIs) to create a training and test data set. Features of obstacles are extracted using convolutional layers and pooling, with RPN and CNN to identify target areas. The results show a detection precision of 99.2% for cars nearby and 87.3% in remote situations. Techniques like Dropout and DropConnect have improved identification, increasing accuracy by 3%. However, CNN provides an accurate and effective solution for detecting and identifying obstacles, surpassing traditional methods and improving road safety. On the other hand, the authors of the research [21] presented an improved approach for real-time vehicle detection utilizing a multi-scale CNN (MSCNN) combined with a high-precision vector prediction mechanism (HPVP). This method focuses on overcoming the limits of existing techniques, such as YOLOv3-tiny and YOLOv3, in terms of precision and computation time. The methodology is tested on three data sets - KITTI VD, AUTTI, and CrowdAI - obtained with a monocular camera mounted in the front of a vehicle. Experimental results show a significant improvement compared to YOLOv3 and YOLOv3-tiny, including a reduction in false positives and false negatives, and a positive impact of PVHP on vehicle detection accuracy. In sum, MSCNN with PVHP provides better accuracy while ensuring competitive compute times, which is advantageous for real-time vehicle detection applications, especially on CPU-based platforms, with potential implications for

embedded environments. Research [22] presents a comparative analysis of the performance of several In-depth learning-based vehicle detection methods, focusing on their effectiveness in road driving scenarios. Models evaluated include SSD, RetinaNet, YOLOv3, R–FCN, and Faster R–CNN, which are driven and tested on the KITTI dataset, which contains real-life images of driving scenes with various lighting, weather, and traffic conditions. The study methodology includes training models with CNN, detection and classification layers, as well as data preprocessing techniques such as magnification to improve generalization. Performance is measured in terms of recall rate, accuracy, average accuracy, and number of frames per second processed (FPS). Results show that RetinaNet excels in detection accuracy while maintaining reasonable detection speed, whereas YOLOv3 and SSD offer higher speeds but lower accuracy. In conclusion, although two-stage methods like RetinaNet offer better accuracy, it is considered the most suitable for autonomous driving applications due to its balance between precision and speed, while future research should aim to develop lighter and faster models for the real-time requirements of autonomous driving systems. Scientists in another study [23] focus on the development of models for effective road detection of objects, with a focus on detection in various weather conditions. The methodology is based on deep neural networks, specifically the YOLOv8 algorithm, combined with a learning transfer to train models with an extensive data set, the Canadian Vehicle Dataset (CVD), including annotated images captured in Quebec, Canada, under different weather conditions. Results show a significant performance improvement when the model is driven on mixed data sets, including the CVD and RoboFlow datasets, with an accuracy of 62.90% for vehicle identification, Pedestrians, and traffic lights. In summary, this study establishes a solid basis for the development of universal models to identify and categorize road objects, with implications for autonomous environments and future road regulations. Study authors [24] propose a new method of detecting pedestrians utilizing the YOLOv5 model, which is famous for its real-time performance. The challenges of pedestrian detection include concealment, low resolution, and lighting variations, factors that make detection difficult for autonomous driving systems. The study method includes data preparation and annotation, training of YOLOv5 model, and performance evaluation. The model showed an accuracy of 75% in daylight and 85% at night, but also displayed a significant number of false negatives (11 in each scenario), highlighting a gap in full detection. The study concludes that, despite the promise of the YOLOv5 model, improvements are needed to reduce false negatives and improve overall accuracy. Contributors in the study [25] also aim to develop and evaluate different methods for the automatic detection of animals in urban environments. Methods used include RetinaNet, YOLOv3, and Faster R–CNN object detection models, as well as segmentation, tracking, and contour detection techniques. To adapt these techniques to specific environments, researchers use transfer learning and the adjustment of models on data sets such as COCO and VOC. The dataset includes mainly animal species found in Canada, and a portion has been enriched with synthetic data to include animals in various artificial backgrounds. The results highlight the need to improve models to manage complex and diverse environments better and to extend synthetic data generation techniques for better coverage of real-world scenarios. Limitations include the need for more specific data and the difficulty of adapting pre-trained models to different environments. The code and data are made available to encourage continued development and research in this area. Furthermore, scientists in article [26] aim to develop and validate an advanced deep-learning framework for real-time recognition of pedestrians and vehicles. The framework uses the most recent object detection methods, including YOLOv8 and RT-DETR versions. The study dataset includes 1, 142 annotated images with six classes of objects (bicycle, bus, car, motorcycle, pedestrian, truck), extended to 3,082 images through various magnification techniques. The results show that YOLOv8 Large outperforms other models, with a 0.909 mAP for pedestrian detection and an inference speed of 40 FPS, compared to 25 FPS for RT-DETR. This study highlights the potential of this framework for traffic monitoring, although further research is needed to extend the framework's capabilities to even more varied and complex scenarios. In addition, the authors of the study [27] proposed a new architecture for remote pedestrian detection, combining GAN with a SSD to enhance the recognition of blurry and Tiny objects in videos or images at a distance. Experiments conducted on the CIFAR-10 dataset demonstrate that DCGAN + SSD architecture boosts detection rates from 35.5% to 80.7%, particularly for small and distant objects. However, challenges remain, including the need for testing in diverse contexts and adapting to larger datasets. Further optimizations are necessary to reduce processing time for real-time applications in smart urban environments. In the study [28], The authors tackled the issue of vehicle-animal collisions on motorways by proposing a cost-effective solution using computer vision techniques. Their method employs HOG descriptors and cascading classifiers to identify animals in images and videos. It achieves an 82.5% detection accuracy based on a dataset of over 2,200 images, effective at speeds up to 35 km/h. However, the system's detection range is limited to 20 m when stationary and requires improvements, such as high-resolution cameras or radars, to enhance night performance and extend detection range; it can also be adapted for other animal types with further training. In the study [29] contributors offer a method to enhance pedestrian classification in images, even when parts of pedestrians are obscured (occluded) or the scenes are complex. The approach suggests a CNN model from scratch, ResNet50 scratch trained, and four transfer learning models: VGG16, GoogLeNet (InceptionV3), ResNet50, and MobileNet. This study utilized four datasets: INRIA, PSU CVC05, and the WU dataset. The experimental results demonstrate that six transfer learning models achieved the following classification accuracies using data from Southern Thailand (PSU dataset): 65.2% for the end-to-end CNN, 92.92% for the scratch-trained ResNet50, 97.15% for the pre-trained VGG16, 94.39% for the pre-trained InceptionV3, 90.43% for the pre-trained ResNet50, and 98.69% for the pre-trained MobileNet. In terms of using CNN models, another approach is presented in the paper [30], an adaptive method for detecting front vehicles under complex weather conditions, such as rainy and snowy days. This paper introduces a cascade vehicle detection method based on CNN. Initially, LBP, Haar-like features, and HOG are extracted. These features are then subjected to Principal Component Analysis (PCA) for dimensionality reduction, followed by feature fusion to define the detection range. A Support Vector Machine (SVM) classifier is then applied to filter the detection range using the fused features. The filtered detection boxes are input to CNN for final detection, forming the basis for cascade detection. The effectiveness of this approach is validated through experiments using datasets from BDD (Berkeley DeepDrive) and Udacity, demonstrating a precision of 98.69% under favorable driving conditions. Additionally, when tested in more complex driving scenarios, the cascade detection method reduces the false detection rate by 1.37% and the omission rate by 0.7%, indicating improved robustness. Compared to other leading vehicle detection methods, the proposed cascade detection algorithm outperforms with superior accuracy and robustness. To enhance object detection performance, a custom dataset was used with an accuracy metric of 72%. The approach combines classical and deep learning techniques for effective object classification and detection [31]. For the classical detector, HOG features are extracted from pedestrian images, then normalized and used to train an SVM classifier. On the deep learning side, a CNN model based on AlexNet architecture is employed. Images are preprocessed to align with the CNN's input dimensions, incorporating standard procedures like resizing, normalization, and augmentation to optimize model performance.

2.2. Comparative study

The comparison of research results enables an evaluation of the effectiveness and diversity of approaches to the detection and classification of mobile barriers. To do this, a comparative table was developed to provide a structured overview of the main characteristics and results of the research examined, it is crucial to keep in mind that to avoid a long summary table, we provide only the most accurate approach per each paper, as highlighted in the related work analysis a single paper may involve multiple datasets and models, which means more than one approach. Therefore, in Table 1 we are keeping only the most accurate approach. This table summarizes key elements of the studies reviewed. The "References (refs)" column contains study citations for specific references. The "Year" column indicates the publication year of each study. The "Objective" column shows the primary objective pursued in each search. The "Methods" column describes the methodological techniques and approaches used. The "Dataset" column provides detailed information on the data sets used, including their name, size, preprocessing applied, and defined classes. Finally, the "Results" column presents the performance achieved using abbreviations such as Acc for Accuracy, Pr for Precision, F1 for F1-Score, R for Recall, AP for average precision, mAP for mean average precision, AP for average precision, and AUC for Area Under the Curve. By examining this table, it is possible to identify the dominant similarities, differences, and trends between studies, thus facilitating a comparative assessment of mobile obstacle detection and classification techniques:

Table 1
Related work summary.

Refs	Year	Goal	Dataset				Method	Results
			Name/Description	Size	Preprocessing	Classes		
[9]	2019	Pedestrian detection	INRIA pedestrian dataset	10,000	Data augmentation	Pedestrians, No-Pedestrian	HOG, LBP, XGBoost	Pr = 97.00 %, R = 97.00 %
[10]	2017	Pedestrian detection	KITTI	14,999	Standardization scaling	Pedestrians	R–CNN and RPN, Skip pooling	mAP = 76.3 %
[11]	2021	Pedestrian and vehicle detection	KITTI	14,999	–	Pedestrians Vehicle	Faster RCNN (ResNet 50)	AP = 58 %
[12]	2018	Pedestrian detection	Pedestrian images	4,330	Data alignment	Pedestrians	CNN, AdaBoost	Pedestrian detection rate: 9 %
[13]	2020	Pedestrian detection	ZUT (images thermiques en 16 bits)	NS	Normalization, data augmentation	Pedestrians Cyclists Vehicle	YOLOv3	mAP = 89.1 %
[14]	2020	Pedestrian and vehicle detection	Pedestrian and vehicles images	1,518	Image resizing	Pedestrians Vehicle	Pednet	Pr = 90.92 %
[15]	2022	Pedestrian detection	COCO (training)	NS	–	Pedestrians	Faster-RCNN	AP = 39.4 %
[16]	2023	Pedestrian detection	VOC2014	NS	Rain-to-dry conversion using Pix2Pix GAN	Pedestrians	YOLOv3	AP = 74.38 %
[17]	2019	Animals' detection	COCO & Google open images	20,000	–	Animals	YOLOv3	mAP = 78.00 %
[18]	2021	Pedestrian and vehicle detection	KITTI	14,999	Remove train annotations and merge some labels in BDD	Pedestrians, vehicle	YOLOV4-5D	mAP = 87.02 %
[19]	2017	Pedestrian and vehicle detection	Indian road images	NS	–	Pedestrians, vehicle	Faster R–CNN, Caffe framework, ZF net pre-trained.	mAP = 97.42 %
[20]	2019	Pedestrian and vehicle detection	PascalVOC 2007	2 GB	–	Pedestrian, vehicle	CNN	mAP = 75.90 %
[21]	2021	Vehicles detection	KITTI	14,999	–	Vehicle	MSCNN, MSCNN_PVHP	AP = 82.23 %
[22]	2019	Vehicles detection	KITTI	14,999	–	Vehicle	RetinaNet,	AP = 89.83 %
[23]	2024	Obstacle detection	Roboflow, CVD	30,000, 8,388	Resizing, and annotating images	Vehicle, Pedestrian, Traffic signs	YOLOv8	Pr = 62,90 %, R = 45.30 % mAP = 47.50 %
[24]	2024	Pedestrian detection	Captured images of pedestrians	Non specified	Image annotation	Pedestrians	YOLOv5	Pr = 75 %
[25]	2020	Animals' detection	Captured images of animals	Non specified	Image annotation	Animals	RetinaNet, YOLOv3 Faster RCNN	Good precision and speed
[26]	2024	Pedestrian and vehicle detection	Extracted from traffic surveillance videos	1,142	Image annotation, data augmentation	Pedestrians, vehicles	YOLOv8l	Pr = 88.40 %, R = 86.10 %, mAP = 90.90 %
[27]	2018	Pedestrian detection	CIFAR-10	60,000	Image enhancement with DCGAN	Pedestrians, Vehicle	DCGAN + SSD	Detection rate from 35.5 % to 80.7 %
[28]	2017	Animals' detection	Animals' dataset	2,200	Image extraction. Enhancement.	Animal (cow)	HOG + cascading classifier	Pr = 82.50 %
[29]	2023	Pedestrian detection	INRIA, PSU CVC05, and the WU dataset	4,415	Data augementation	Pedestrians	CNN, MobileNet	mAP (CNN) = 44 %, acc (CNN) = 65.2 % mAP (MobileNet) = 78 %, Acc (MobileNet) = 98.69 %
[30]	2021	Vehicle detection	BDD + Udacity	NS	LBP, HOG, Haar-like feature, PCA	Vehicles	CNN + SVM	Pr = 98.69 %
[31]	2020	Pedestrian detection and classification	Custom dataset	NS	HOG	Pedestrian	CNN (AlexNet)	Acc = 72 %

By examining objectives, methods, data sets, and outcomes, Table 1 identifies the most effective approaches and potential gaps in current research. This comparative analysis provides a solid basis for assessing progress and guiding future research in the field of road detection and categorization. The study of recent work on mobile obstacle detection highlights significant progress in this important field for road safety. Among the most frequently discussed obstacles, pedestrians and vehicles are central. This predominance is due to the need to ensure safety in urban and road environments, where these obstacles constitute an immediate danger. Although less explored, animal detection is also present in some research, particularly in systems for specific environments. In terms of data sets, the KITTI [7] package is the reference. Its widespread adoption is based on the richness and diversity of images it offers, covering a wide range of road scenarios. Other datasets, such as Pascal VOC [8] and COCO [32], are also widely used because of their detailed annotations and ability to train models in various contexts. On the detection model side, deep learning techniques dominate current research. Faster R–CNN variants, especially with architectures like ResNet 50, ResNet 101, and Inception-ResNetV2, are popular for their high accuracy. SSD (Single Shot Multibox Detector), with MobileNetV1, MobileNetV2, and InceptionV2-based versions, is also highly prized for its speed and real-time efficiency. The YOLO (You Only Look Once) [33] is one of the most widely used models, especially with the YOLOv3, YOLOv4, and more recently YOLOv8 versions, which continue to improve real-time performance while maintaining remarkable precision. Other models such as RetinaNet, R–FCN, and hybrid approaches like HOG coupled with cascading classifiers or XGBoost are also adopted for specific use cases. Specialized networks such as Pednet, Multiped, and RT-DETR are particularly effective in detecting targeted obstacles. Some models are distinguished by their exceptional performance. For example, Faster R–CNN applied to Indian road-specific datasets, has achieved a 97.42 % mAP, making it one of the best-performing architectures for complex road environments. On the other hand, the association of HOG and XGBoost has proven its effectiveness in pedestrian detection, with an AUC of 99.13 % on the INRIA dataset. Finally, YOLOv8l and YOLOv8m variants of YOLOvv8 have recorded impressive performance, reaching a 90.9 % mAP for vehicle and pedestrian recognition with an accuracy value of 88.40 %. In conclusion, recent research on the detection of mobile obstacles shows clear progress in utilizing deep learning techniques, with increasingly efficient accuracy and speed. The work highlights a particular focus on pedestrians and vehicles, while gradually exploring other types of obstacles. These promising developments contribute to improving road safety by providing robust detection systems that are adapted to a variety of environments. The analysis of state-of-the-art works in Table 1 reveals that most studies have employed solid and innovative approaches as well as employing various types of datasets. However, there remains room for improvement due to moderate metric results. Additionally, the datasets used in these comparisons primarily focus on specific vehicles, pedestrians, or animals—or a combination of only two. In response, our approach introduces a new dataset that consolidates all possible classes found in previous research, grouped into three main categories: vehicles (including cars, trucks, motorcycles, etc.), animals (such as cows, dogs, and cats), and pedestrians. This comprehensive group aims to improve accuracy and versatility in real-world applications.

2.3. State of art descriptive analysis

We'll be covering descriptive analytics of the state of the art we provided in the 2.1 section, as always, we start with a higher overview and then drill down to a deep dive. In general, we have 20 papers split based on the task or the goal:

The results in Fig. 1 highlight key areas of research in detection technologies. Both Animals' Detection and Detection of Pedestrians and Vehicles have 3 references each, representing 15% of the total. Obstacle Detection During Driving is significant with 5 references, accounting for 25%, which underscores its importance in driving contexts. The area with the most attention is Pedestrian Detection, featuring 8 references and making up 40% of the total, reflecting its vital role in safety and automation. Pedestrian Detection is among the most crucial and high-stakes tasks for autonomous systems, as it focuses on preventing collisions with potentially life-threatening consequences. If a system fails to detect a pedestrian, the impact can be severe. Pedestrians, unlike other objects, lack any protective barriers, making them highly susceptible to serious or fatal injuries, especially in vital areas like the head, spine, and chest. This risk is heightened by the often unpredictable behavior of pedestrians, making accurate detection across various conditions—such as low light, adverse weather, or crowded areas—vital for dependable

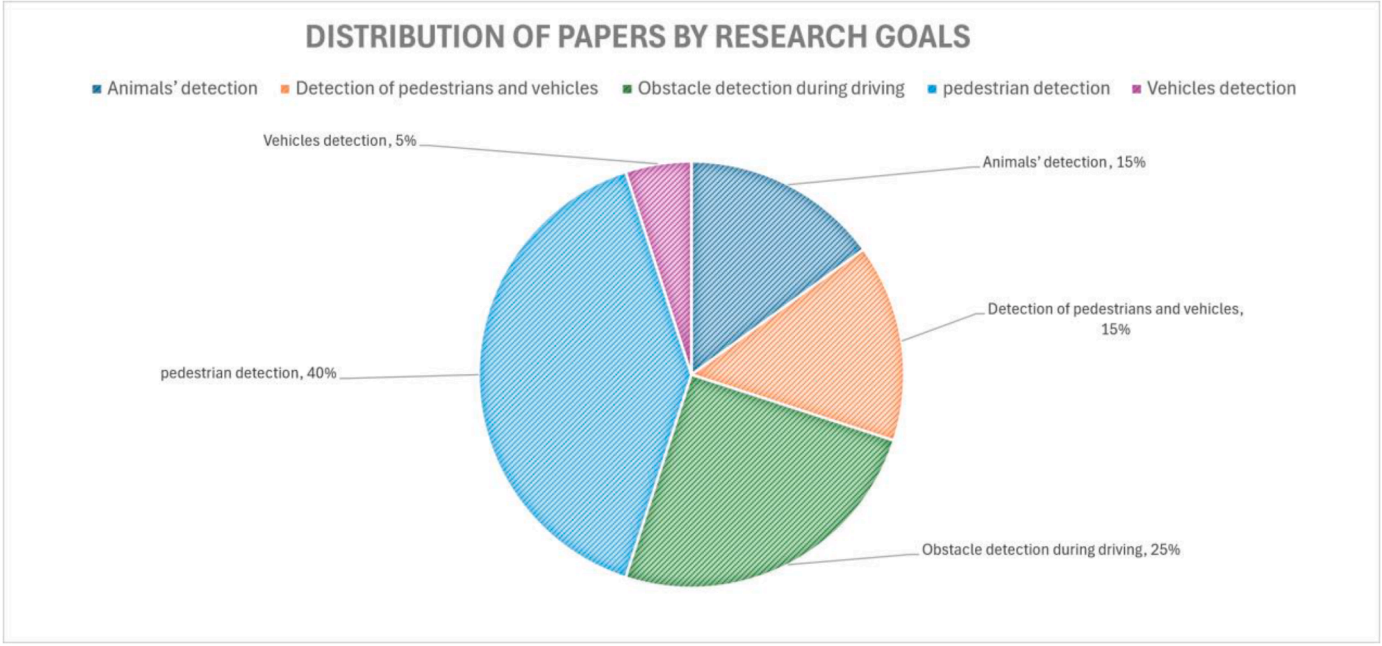


Fig. 1. Papers distribution by paper aims.

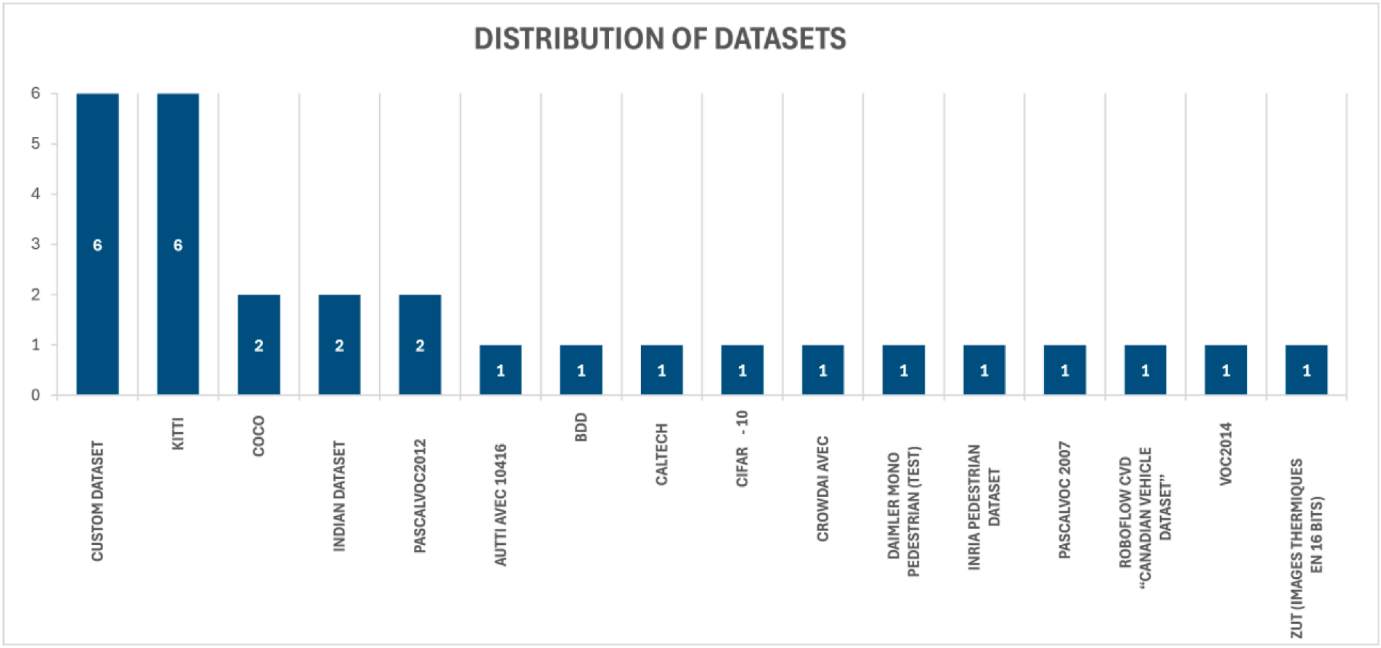


Fig. 2. Datasets distribution across papers.

automated systems. Conversely, Vehicles Detection is the least explored, with only 1 reference, which is just 5%. This distribution shows a strong focus on pedestrian detection while revealing a lack of research in vehicle detection.

Figure 2 shows the dataset distribution across the analyzed papers reveals a diverse range of datasets, each tailored to specific research objectives. Datasets such as AUTTI, BDD, Caltech, CrowdAI avec, Daimler Mono Pedestrian (Test), PascalVOC 2007, Roboflow CVD, VOC2014, and ZUT highlight the specialized applications in autonomous driving and object detection, focusing on unique aspects like weather variations, lighting conditions, and thermal imagery. These datasets vary significantly in size, with examples including BDD's 100,000 images, Caltech's 250,000 images, and CVD's 8,388 images. Additionally, they target different types of objects: some are dedicated to animals, pedestrians, or vehicles, others to a combination of two of the three objects. However, no dataset comprehensively includes animals, pedestrians, and vehicles. To fix this gap, we suggest a custom dataset in this work, combining multiple sources to provide a more comprehensive solution. Notably, the Custom Dataset and KITTI stand out as widely adopted resources, valued for their adaptability and thorough coverage of diverse scenarios, especially in autonomous driving. Established datasets like PascalVOC2012, COCO, and the Indian Dataset remain relevant, supporting foundational object detection and classification tasks. This varied selection underscores the need for multiple datasets that address specific environmental conditions, object types, and scene

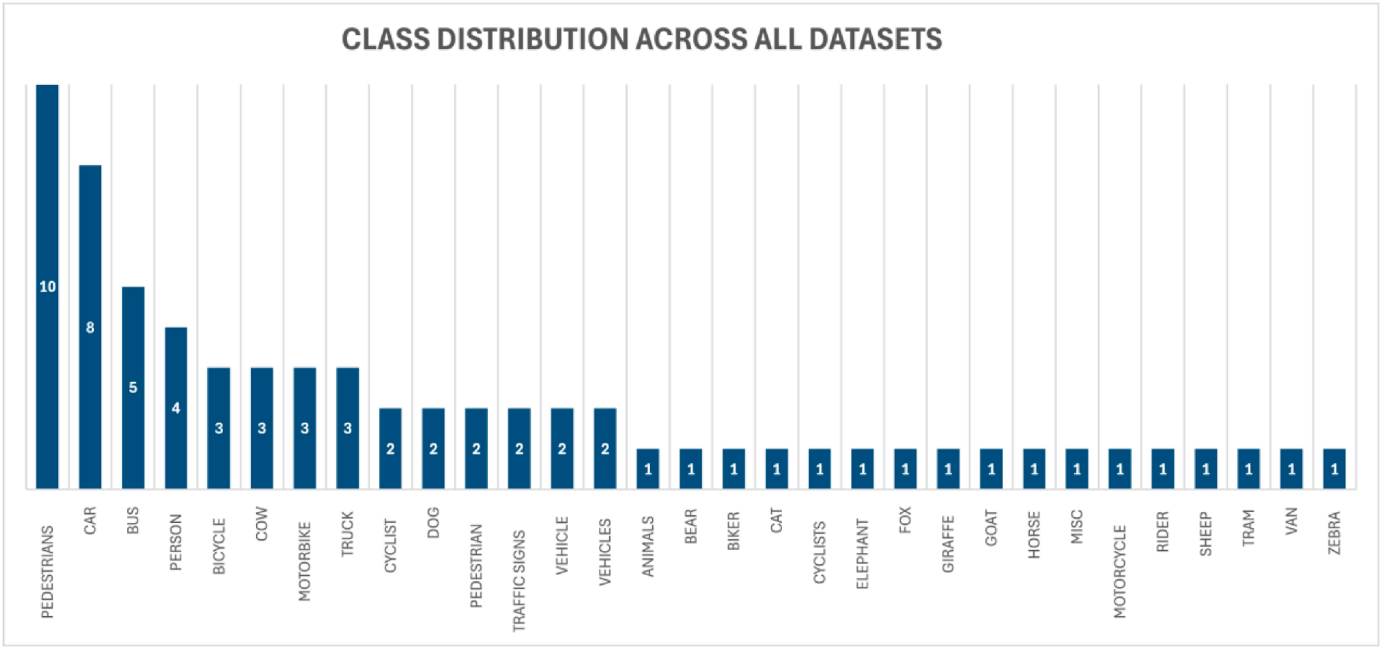


Fig. 3. Class distribution.

complexities, highlighting the necessity for context-specific data in thorough research analysis.

The class distribution across all datasets in our state-of-the-art discussion highlights the frequency of various objects. Pedestrians are the most referenced category, as shown in Fig. 3 with a total of 10 mentions, followed by Cars with 8 and Buses with 5. Other notable classes include Persons (4), Bicycles and Motorbikes (3 each), and Dogs (2). Several animal categories, such as Bear, Cat, Elephant, Fox, Giraffe, Goat, Horse, Sheep, and Zebra, each appear once, indicating limited representation. This distribution emphasizes the research's strong focus on pedestrian and vehicle detection.

Figs. 4 and 5 show that the Faster R–CNN Framework with a pre-trained Caffe ZF Net achieved the highest mAP of 97.42, demonstrating exceptional accuracy in object detection. Other strong performers include YOLOv8l (90.90), RT-DETR-x (89.20), and YOLOv3 (89.10), making them robust choices for tasks that demand high precision. In contrast, methods like Cascade R–CNN (68.00), RetinaNet (69.00), and YOLOv4-5D (70.13) perform reliably. Still, they are less precise, while SSD and its variants (e.g., SSD InceptionV2 and SSD MobileNetV1) show considerably lower performance, with mAP scores around 22.68. This significant disparity highlights challenges in accuracy, likely due to architectural and training limitations in the SSD framework. Additionally, Faster R–CNN variants, such as those using InceptionV2 and ResNet 50, fall within a lower mAP range, suggesting potential limitations in configurations or compatibility with specific detection contexts.

While YOLOv3 shows variability, with mAP values from 31.60 to 89.10, its tiny variant balances speed and accuracy (mAP 71.50), which is beneficial for real-time applications despite a trade-off in precision. The versatility of the Faster R–CNN framework is evident in its consistent, mid-tier performance across different configurations. These findings emphasize the need for our study's approach: grouping diverse datasets and leveraging models like YOLO to achieve a balanced solution. By addressing these challenges, particularly the trade-offs in precision and speed, our work aims to optimize object detection accuracy across varied real-world scenarios.

3. Methodology

This section details the rigorous steps taken to implement our approach for mobile obstacle detection. First, we collected rich and diverse data sets to ensure the representativeness of different types of obstacles. Then, data was pre-processed to improve the quality of inputs and optimize model performance. Subsequently, the data was merged, combining several sources to strengthen the robustness of the drive. Data was then split into training, validation, and testing sets to allow for an unbiased assessment of the model. Finally, our proposed OD–CNN–18 Layers CNN approach was trained on this data before being evaluated using various metrics, thus allowing the effectiveness of mobile obstacle detection to be accurately measured. Fig. 6 presents the methodology adopted for the second part of this project, illustrating the key steps and processes involved from data collection to final model evaluation.

3.1. Data collection

This is a fundamental step in the accurate detection of dynamic obstacles such as pedestrians, vehicles and animals. To ensure full coverage of possible scenarios, we have adopted a rigorous approach, gathering data from various sources. Images were captured in a variety of real-world environments, including urban and rural areas, at different times of the day and under various weather conditions. This collection process has been designed to reflect the diversity of situations encountered on roads, ensuring that the data obtained is representative of the challenges posed by detecting mobile obstacles in various contexts.

3.1.1. Pedestrian and vehicle data

As part of our study on mobile road obstacle detection, we selected the KITTI [7] data set for pedestrians and vehicles. The KITTI dataset [7] is among the most widely used datasets for image treatment research, especially in the field of autonomous driving. It was created by the KIT in Germany and consists of various sub-sets designed to evaluate different computer vision tasks. The KITTI data set includes images captured by cameras installed on a vehicle, as well as data from LiDAR and GPS/IMU sensors, providing a rich variety of multi-sensory data. Annotations are available for objects like vehicles, pedestrians, cyclists, and more, making it a valuable resource for algorithm development and validation in the domain of self-driving vehicles. The dataset is available on the official KITTI website [34]. It contains more than 14,999 images (7,481 and 7,518 images for train and test respectively), and the images are a size of 1,242 × 375 pixels. Annotations are provided as text files (txt) containing the enclosed box coordinates, object class, orientation, and other attributes. Annotated objects include vehicles, pedestrians, cyclists, and other obstacles, it also covers urban, rural, and highway

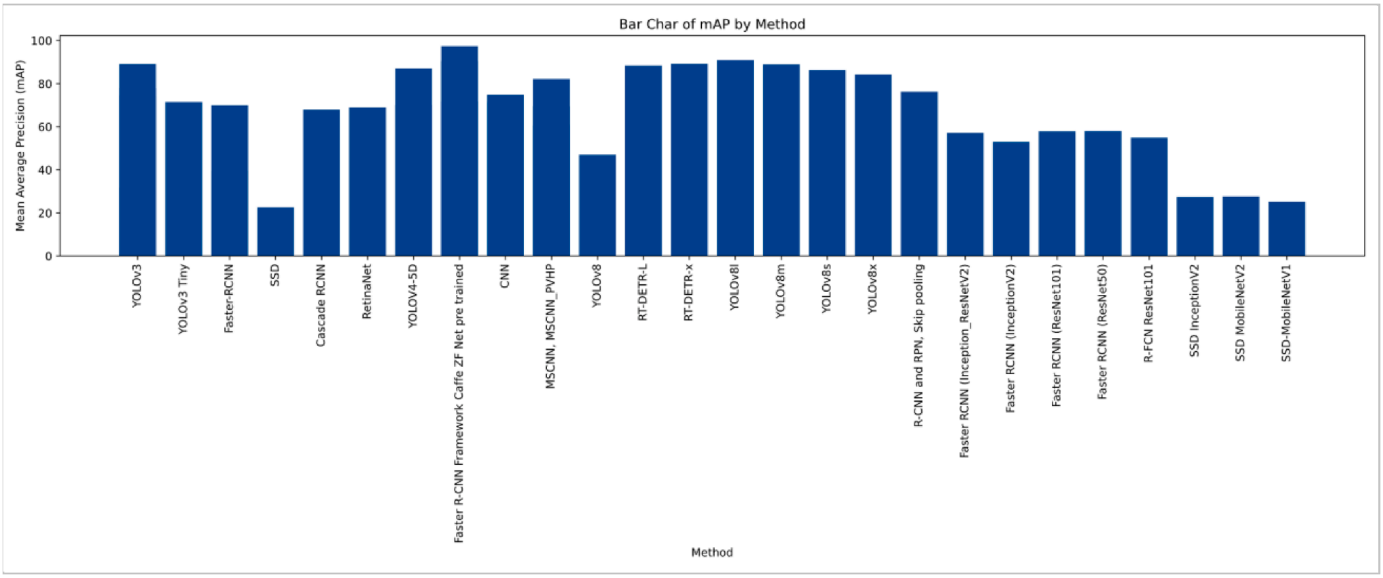


Fig. 4. mAP Bar chart by detection method.

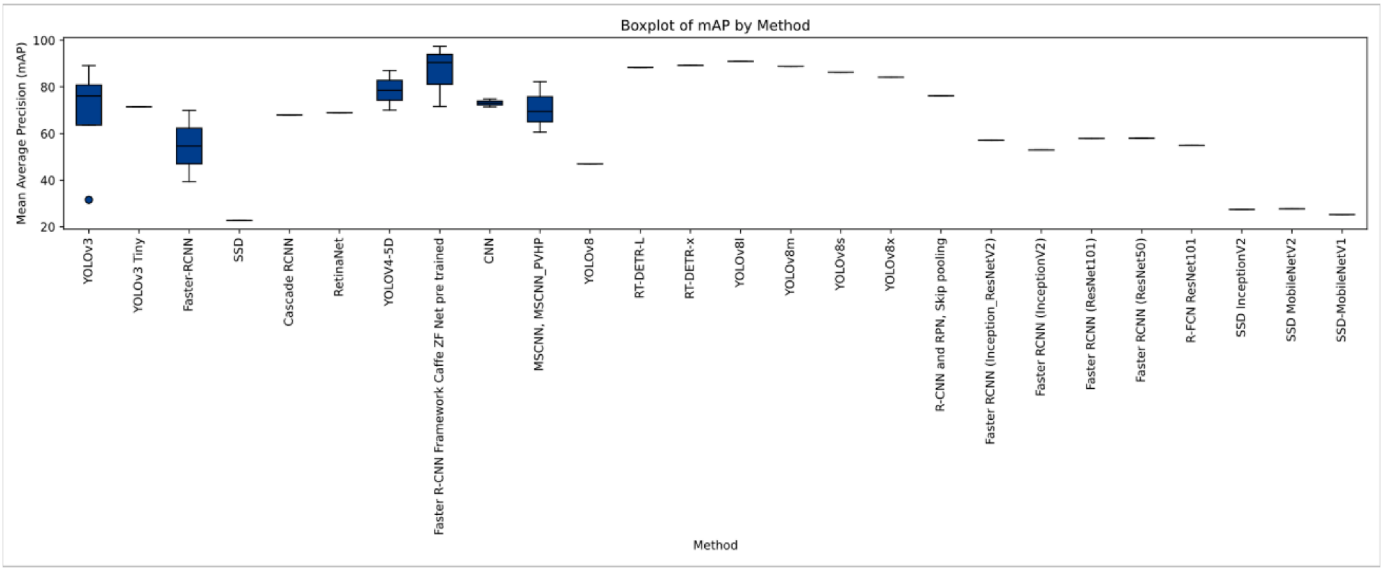


Fig. 5. mAP Box Plot by Method.

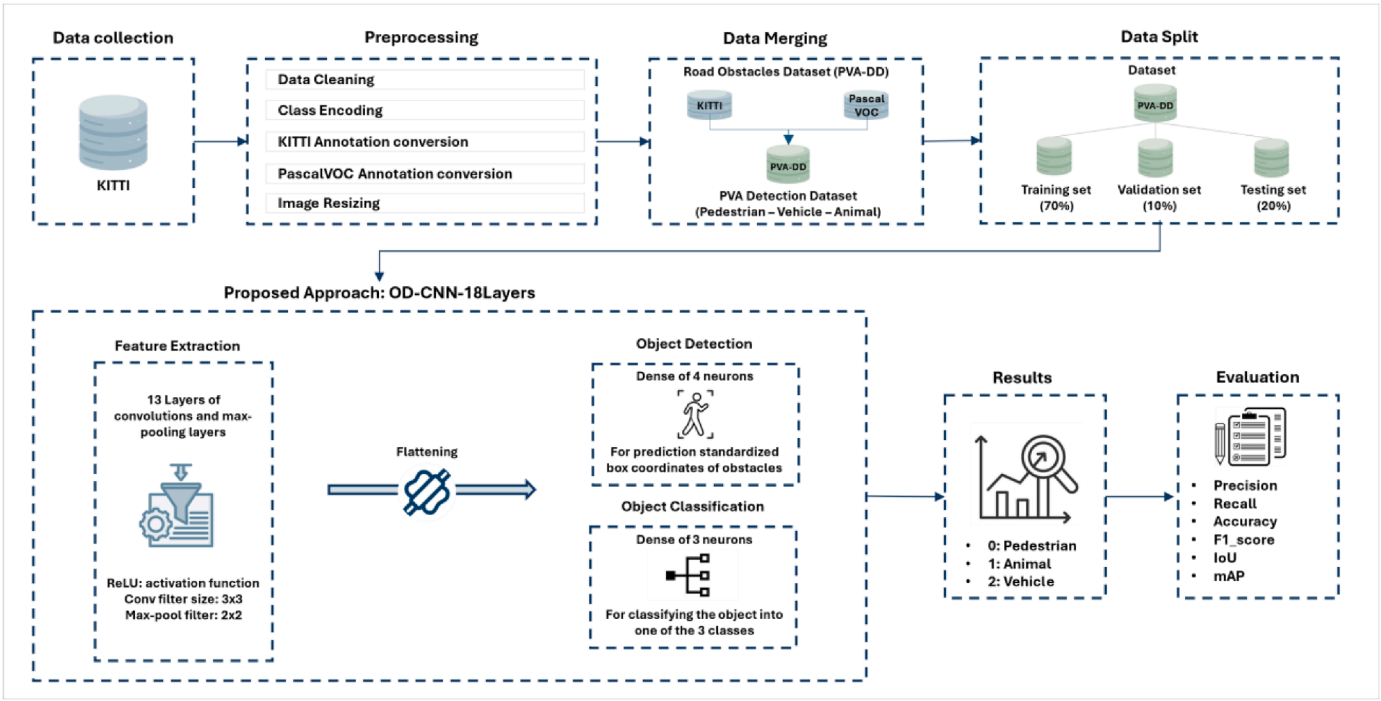


Fig. 6. Methodological approach.

```
Pedestrian 0.00 0 -0.20 712.40 143.00 810.73 307.92 1.89 0.48 1.20 1.84 1.47 8.41 0.01
```

Fig. 7. KITTI annotation example.

environments with variability in weather circumstances, period of day, and traffic.

The KITTI dataset basically includes a comprehensive collection of images that were taken from a moving vehicle, covering a range of urban, rural, and highway areas. This dataset covers different scenarios like pedestrians walking/crossing the streets, cars on the road, buses, motorcycles, and even cyclists navigating through traffic, introducing a wide range of use cases for training and testing detection models.

3.1.2. *Animals' data*

For animals, we have chosen the data set named "PascalVOC2012" [8]. The PascalVOC2012 data set is one of the most important and influential references, particularly for object detection, object segmentation, and image classification tasks. It was developed as part of the "Visual Object Classes Challenge" (VOC) organized by the PASCAL Network, a research network supported by the European Union. This dataset has been the basis for many advances in computer vision

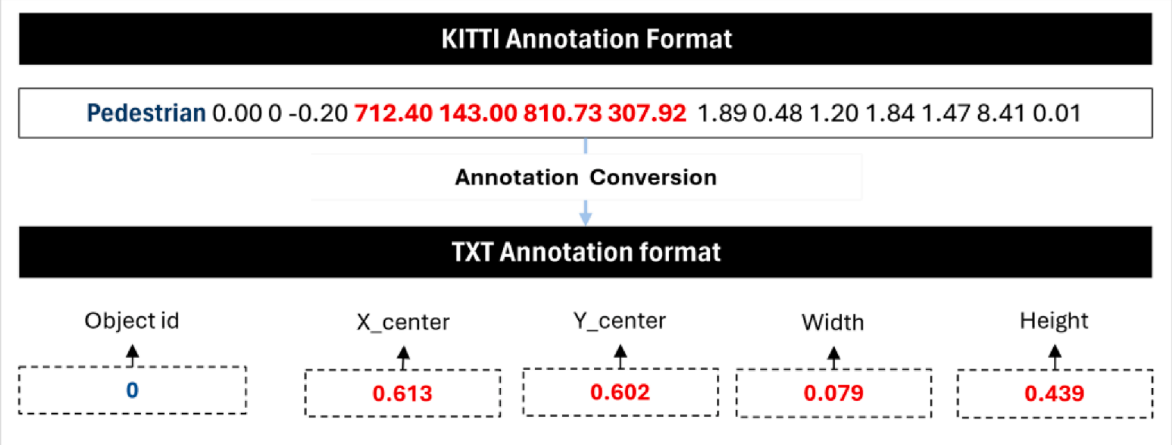


Fig. 8. Process for converting KITTI annotations to TXT (Pedestrian).

algorithms and has been widely used to evaluate model performance in realistic scenarios. PascalVOC2012 [8] consists of various images, captured in real-world conditions, including 20 classes of objects divided into different categories such as animals (e.g. dog, cat, horse), vehicles (e.g. car, motorcycle, airplane), furniture (for example, chair, table), and people. In total, the dataset contains approximately 11,530 images, with more than 27,450 annotated objects for detection and approximately 6,929 annotated images for semantic segmentation. Annotations are accurate and include enclosing boxes for object detection, pixel-by-pixel segmentation masks for semantic segmentation, and labels for image classification.

The PascalVOC2012 dataset provides a collection of images comprising a wide range of animals, such as birds, cats, cows, and dogs … these images are annotated with pixel-level segmentation, bounding boxes, and object class labels, making it detailed for analysis and model training for animal classification and detection tasks.

3.2. Data preprocessing

In this phase we'll see the preprocessing techniques we used to allow the data to be prepared to be used by the model, this allows us to guarantee the quality of our dataset and ensure the optimal performance for the detection and classification of our objects. Since we had two different datasets, we present the preprocessing for each one as follows:

3.2.1. Pedestrian data preprocessing

For pedestrian data, we used the "KITTI" dataset, and since it includes several classes of objects. The first step in our process was to select only pedestrian data. This selection allowed us to focus on a relevant subset of the data, facilitating more focused analysis. After the selection of pedestrian data, we started the data-cleaning phase. This step consisted of deleting the empty annotation files and associated images, as these files did not contain any objects of interest. By eliminating these irrelevant elements, we have ensured that the data set is composed only of valid and useful files, which helps optimize the accuracy and reliability of detection models. Once the cleanup was complete, we proceeded to encode the classes. The annotations of "KITTI" were initially in the form of categorical class names, such as "Pedestrian". To make these annotations compatible with the entry of our model, we converted these categorical class names into integers. In our case, the name of the class "Pedestrian" has been encoded as an integer 0. This digital transformation simplifies data processing and allows the model to efficiently manipulate information about pedestrians. We then moved on to the annotation conversion pase. The "KITTI" annotations are generally as shown in Fig. 7:

Each element of this annotation represents:

- Class: "Pedestrian" Name of the annotated object class.
- Truncation: "0.00″ Indicator of the visible part of the object in the image (0 = Completely visible).
- Occlusion: "0″ Occlusion level indicator (0 = not obscured).
- Difficulty: "-0.20″ Detection difficulty level indicator (no value standardized, often used for other information).
- Box Enclosing Contact Information: "712.40 143.00 810.73 307.92″ Coordinates of the upper left corner (712.40, 143.00) and lower right corner (810.73, 307.92) of the enclosing box in pixels.
- Dimensions: "1.89 0.48 1.20″ Dimensions of the object (height, width, depth) in meters (values not used in TXT format).
- Other information: "1.84 1.47 8.41 0.01″ Additional information often related to the depth or speed of the object.

We have converted these KITTI annotations into a simpler TXT format, which consists of: «class, x, y, w, h » where:

- Class: Class number (for example, 0 for "Pedestrian").
- x, y: coordinates of the center of the enclosing box, normalized according to the dimensions of the image. For example, (x, y) can be determined as the mean coordinates of the enclosing box.
- w, h: Width and height of the enclosing box, also standardized.

This conversion allows easier integration with detection models using simpler annotation formats. To better understand this conversion process, here is an illustrative example of the transformation of an annotation is provided in Fig. 8, going from KITTI format to TXT format for the "pedestrian" pedestrian object), assuming that the width and height of the image are 1,242 × 375 pixels.

After converting all KITTI annotations to TXT format, we have a folder containing the pedestrian annotations in TXT format. Each line in each file represents the information of an object, namely its class, its coordinates. We then resized the images to a standard size of 644 × 644 pixels. This resizing ensures consistency of the input dimensions for model drive and can improve performance by reducing computational load.

3.2.2. Vehicle data preprocessing

For vehicles, we used the same "KITTI" dataset. We first selected data for the different vehicle categories, namely cars, vans, and trucks. These categories have been grouped under a single class called "Vehicle" to simplify model analysis and training. Then we started the data cleaning phase, where we eliminated the empty or erroneous annotation files and the corresponding images that do not contain objects of interest. This cleaning ensures that only relevant annotations and quality images are retained. After data cleaning, we have transformed object classes into

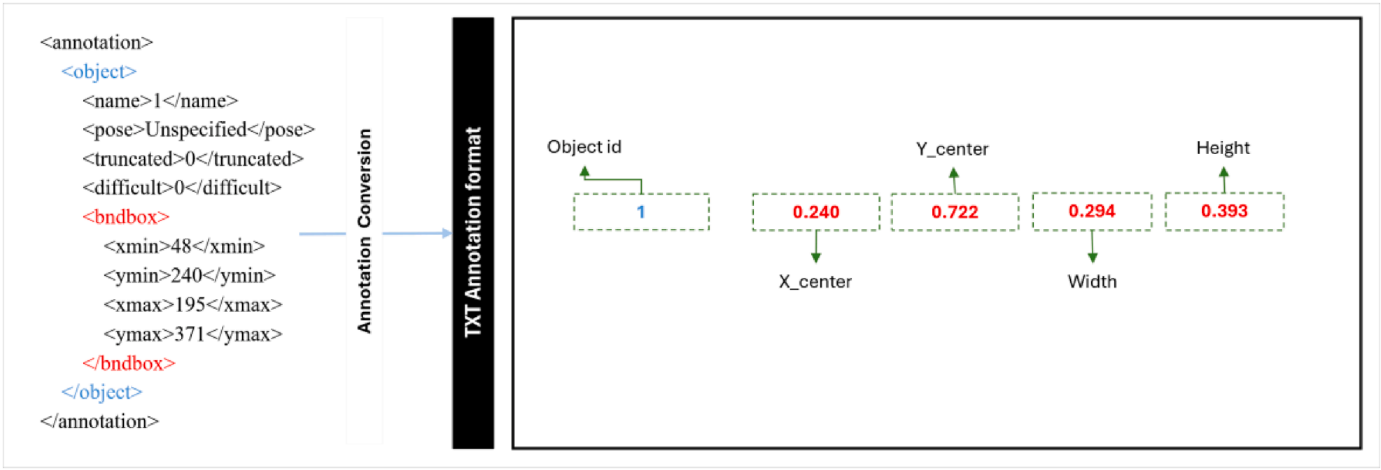


Fig. 9. Process for converting annotations from XML to TXT (Animals).

numerical values. In this case, we assigned the integer "2″ to the class "Vehicle". To clarify this transformation process, we illustrate below an example of conversion from the KITTI format annotation to the TXT format for the object "vehicle". This process is based on an image with dimensions of 1,242 × 375 pixels, allowing to visualize how the annotations are adapted to the new format.

3.2.3. *Animal data preprocessing*

For the animal data, we used the "PascalVOC2012" dataset. To simplify the training of the model, we have grouped these different classes under a single generic class named "Animal". This step simplified the model by limiting the classes to be processed. Once the data selection was done, we started the data cleaning phase, which was a critical step to eliminate all empty annotation files and associated images. These empty files are often the result of images containing no objects of interest or data corruption. By deleting these files, we were able to make sure that the remaining data was relevant and ready for training. After cleaning, we proceeded to encode the classes. Since the VOC Pascal annotations use categorical class names, it was necessary to convert them into a numerical representation that the model could easily interpret. To do this, the class "Animal" has been encoded in a unique integer, specifically in "1". This has helped to standardize the annotations and make them compatible with our processing pipeline. After completing the class encoding, we started the annotation conversion phase. Since the annotations of the VOC Pascal dataset are in XML format, it was necessary to transform them into a simpler TXT (as illustrated in Fig. 9) format that is suitable to our model. This conversion involved extracting the relevant information, such as the class of the object, extracted from the <name> tag, and its coordinates, obtained from the <bndbox> tag (including xmin, ymin, xmax, and ymax values), and then reorganizing them into a format that is more appropriate to our training pipeline. To better understand this conversion process, here is an example of how to transform an annotation from XML to TXT. Suppose the image width and height are 500 × 333 pixels.

After converting all annotations from XML to TXT format, we have a folder containing the animal annotations in TXT format. Each line in each file represents the information of an object, namely its class and coordinates. Once the annotation conversion was complete, we resized the images to fit the standard size of 644 × 644 pixels.

3.3. *Data merging*

After collecting and pre-processing data for the three types of obstacles (pedestrians, vehicles, animals), we merged these selected datasets into one global dataset with multiple classes. This process resulted in a final data set "PVA Detection Dataset" (PVA DD), where "PVA" refers to the three types of obstacles included, namely Pedestrian, Vehicle, and Animal. For a better understanding of the composition of our data set, Fig. 10 illustrates in detail the distribution of images for each type of obstacle. This visualization highlights the number of images associated with each category thus providing a preview of the balance of data within the "PVA Detection Dataset".

Our final data set, titled "PVA Detection Dataset", consists of three types of obstacles, each represented by a significant number of images.

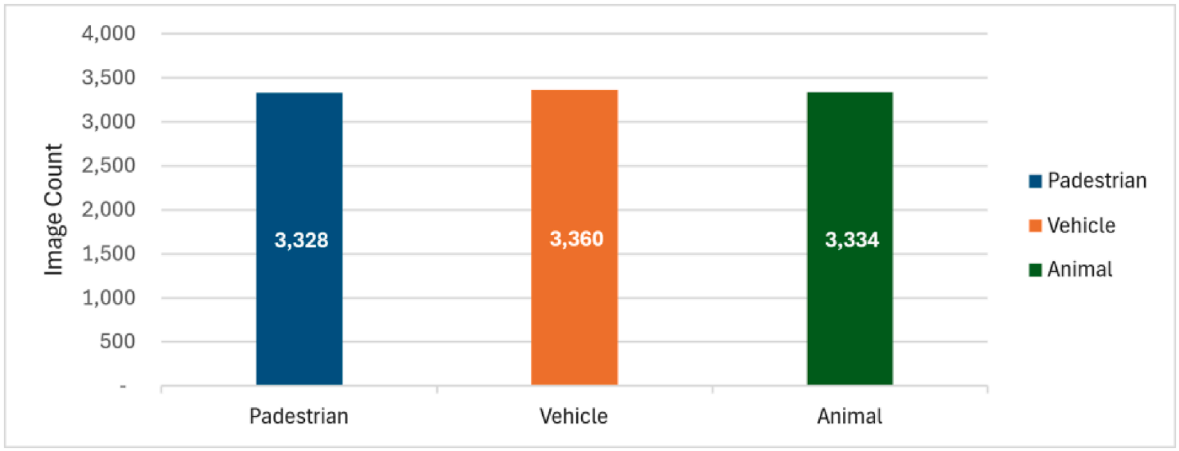


Fig. 10. Image distribution for the data set "PVA-DD".

Table 2
Breakdown PVA-DD data into each subset.

Dataset	Image count	Number of tags	Total
Train	7,014	7,014	14,028
Validation	1,003	1,003	2,006
Test	2,005	2,005	4,010
Total	10,022	10,022	20,044

Specifically, it contains 3,328 images of pedestrians (pedestrians), 3360 images of vehicles (vehicles), 3,334 images of animals (animals). This balanced distribution ensures adequate coverage for each type of obstacle, providing a good starting point for the training of robust and high-performance detection models.

3.4. Data splitting

The "PVA Detection Dataset" was divided into three parts to ensure optimal training and assessment of the model. We allocated 70% of the data to the training subassembly, 10% to the alignment subassembly, and the remaining 20% was reserved for final testing. To examine this distribution in more detail, Table 2 displays the number of corresponding images and labels present in each subset (training, validation, and testing).

3.5. Our proposed CNN model architecture

After dividing our dataset, we moved on to the training phase of models for road obstacle detection and classification. For our approach, we designed a model of a CNN called "OD–CNN–18 Layers (Obstacle Detection–CNN–18-layers). This model comprises 18 powerful layers and is structured to explore and extract complex features from road obstacle images with remarkable precision. Thanks to its depth and specialized layers, the OD–CNN–18Layers can discern subtle details and intricate patterns in images, making it particularly effective for detecting and classifying mobile road obstacles. Each layer of the model plays a key role in transforming raw data into relevant information, allowing for a fine and detailed analysis of various types of obstacles encountered on roads.

The choice of a CNN was driven by our goal to create a model from scratch that is dedicated and customized specifically for the KITTI and PascalVOC datasets. Unlike pre-trained transfer learning models, which are typically designed for general-purpose image classification tasks, our custom CNN is specifically tailored for road obstacle detection and classification. Pre-trained transfer learning models, such as ResNet or Inception, are computationally expensive in terms of both memory and inference time. Since our system is designed for real-time road obstacle detection, we prioritized computational efficiency by developing a lightweight yet effective custom CNN that achieves high performance without the overhead associated with large pre-trained models. Fig. 11 illustrates the architecture of our model.

Our model architecture is as follows:

- **Feature extraction:** The model begins by using several convolutional and max-pooling layers to capture and extract important features from images at different levels. The first series includes convolutional layers with 64, 128, 128, 256, 256, 512, 512, 1,024,

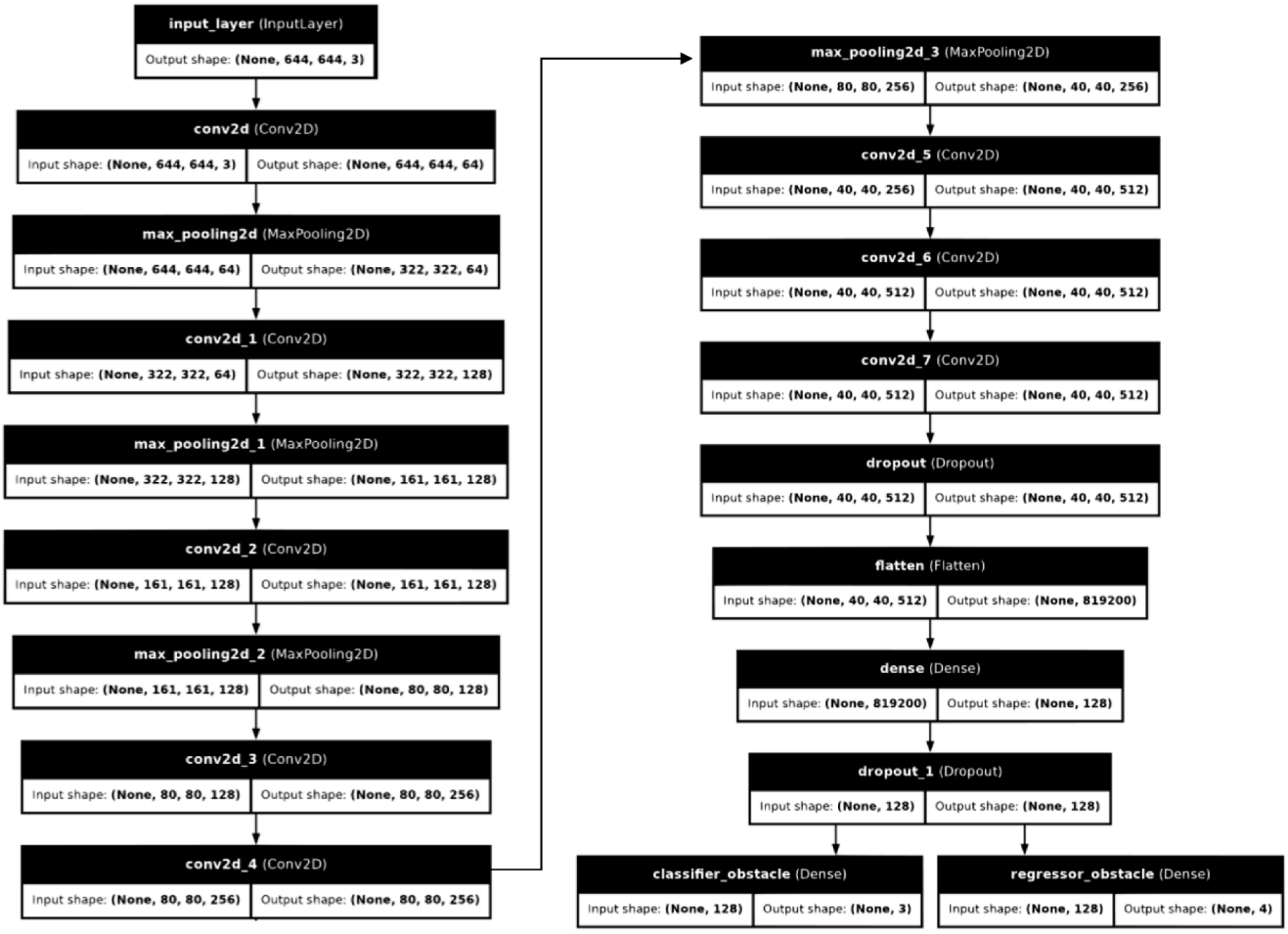


Fig. 11. Obstacle detection–CNN–18-layers.

1,024, and finally 2,048 filters, with a 3 × 3 core size and ReLU activation. Each conv block is followed by a max-pool layer with a 2 × 2 core size, which allows for the gradual reduction of the size of the features while increasing their depth. The last convolution layer uses a 2 dilation to capture details at a larger scale, followed by a 0.3 Dropout layer to avoid overlearning.

- **Model adapter:** Flatten and dense layers: The outputs are flattened and fed to a dense layer containing 128 neurons and ReLU activation. This step is followed by another Dropout layer with a factor of 0.3 to once again reduce the risk of overlearning.
- **Detection and classification:** Obstacle detection: The output of the adapter uses a dense layer with 4 neurons to predict the coordinates of the boxes enclosing obstacles, which allows an accurate regression of dimensions and positions of detected obstacles. Obstacle classification: In parallel, the output of the adapter passes through a dense layer with several neurons equal to the number of classes, 3 in this case. These classes represent the 3 types of road obstacles (Pedestrians, Vehicles, and Animals). The Softmax activation function is used to classify types of obstacles.

3.6. Model compilation

The OD–CNN–18 Layers model is compiled with the Adam optimizer, which is particularly effective for deep learning tasks due to its ability to adjust learning rates in an adaptive manner. For obstacle classification, we use the categorical_crossentropy loss function, which is appropriate for multi-class tasks like ours with three classes (pedestrians, vehicles, animals). For the regression of the coordinates of the encircling boxes of obstacles, the MSE loss function is used to reduce the discrepancy between actual and predicted values.

4. Experiment and results

4.1. Model hyperparameters

For the OD–CNN–18 Layers model, we have implemented the Early Stopping technique to optimize performance and avoid overlearning. Training is planned to run up to 100 times, with Early Stopping monitoring loss of validation to stop the process if no improvement is seen for 10 consecutive times. This strategy allows us to keep the best weights of the model and ensure a good generalization on the unseen data. The model is trained with close performance monitoring to adjust the process in real-time. Table 3 presents all the tuning parameters used in our model pipeline.

Table 3 provides a complete overview of the essential hyperparameters, and configurations used for training our OD–CNN–18 Layers model.

Table 3
Hyperparameters adopted for our model OD–CNN–18 Layers.

Hyperparameters	Value	Description
Dropout	0.3	The dropout rate is applied to reduce overlearning.
Input size	(644, 644, 3)	Dimensions of the input images (width, height, channels).
Batch size	32	Batch size used during training
Convolution kernel size	3 × 3	Kernel size for convolutional layers.
Activation function	ReLU	Used in convolutional layers.
Activation function for the classification	Softmax	Softmax activation function is used for the classification of types of obstacles.
Loss function for classification	Categorical crossentropy	The loss function is used for the classification of types of obstacles.
Loss function for regression	MSE	The loss function is used to predict the coordinates of the enclosing boxes.
Optimizer	Adam	Optimizer employed for training the model.
Learning-rate	0.001	The learning rate is used for weight adjustment.
Epochs	100	Maximum number of times for model training.
Early stopping	10	Number of times without improvement before stopping training.

4.2. Experimental metrics

In this subsection, we will outline the various metrics employed in the results section, as well as the software and hardware environment utilized for conducting our experiments.

4.2.1. Metrics

- *Precision:*

It refers to the quality of good predictions achieved by the model, it responds mainly to the question "What's the rate of all the positive classifications that were accurately identified?", it is calculated mathematically by splitting the occurrence of true positive by the positive predictions (true positive + false positive):

$$\mathrm{Pr} = \frac{\text{classified actual positive}}{\text{all instances classified as positive}} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

FP: false positive, TP: true negative, FP: false positive, FN: false negative.

- *Recall:*

This metric answers this question: "What is the probability of the real positive that was predicted as positive?", it can be calculated using the formula below:

$$\mathrm{Re} = \frac{\text{classified actual positive}}{\text{all actual positive}} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

- *F1 score:*

This metric corresponds to the Mean of the precision and recall providing a balance between the two, it shows the overall effectiveness of the model classification of the positive instances, and it can be calculated using this equation:

$$\mathrm{F1_socre} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \times \mathrm{TP}}{2 \times \mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

- *IoU:*

IoU is one of the metrics to assess the precision of the model in object detection, it shows how well the predicted bounding boxes align with the correct ones (overlap area), and it helps mainly detect correct detections vs incorrect ones. Given P is the surface of the predicted box and A is the actual surface of the box containing the object, IoU can be calculated as:

$$\mathrm{IoU} = \frac{P \cap A}{P \cup A}$$

- *Mean average precision (mAP):*

Table 4
Average performance measures obtained by OD–CNN–18 Layer.

Accuracy	Precision	Recall	F1 Score	IoU	mAP@0.5	mAP@0.5:0.95	MSE
99.5 %	97.1 %	99.5 %	98.4 %	77.8 %	97.6 %	66.2 %	0.016

Table 5
OD–CNN–18 Layers performance metrics by class.

Class	Accuracy	Precision	Recall	F1 Score	IoU	mAP@0.5	mAP@0.5:0.95
Pedestrian	99.4 %	99.9 %	99.4 %	99.6 %	85.2 %	97.2 %	66.7 %
Animal	99.4 %	92.5 %	99.3 %	96.3 %	67.7 %	98.6 %	73.4 %
Vehicle	99.6 %	99.0 %	99.7 %	99.3 %	80.5 %	97.0 %	58.7 %

The AP refers to the surface below the pr-re curve by varying the IoU confidence threshold, the mean average precision is the meaning of these average precisions, it can be calculated using this formula:

$$\mathrm{mAP} = \frac{1}{n}\sum_{k=1}^{n}(\mathrm{AP}_k)$$

where AP_k : class k Avg Pr. And n is the class count.

- *MSE:*

It is utilized to measure the mean squared subtraction between the actual values and the predicted ones in regression task, it responds to how accurate the predictions are compared to the actual data, its formula is as follows:

$$\mathrm{MSE} = \frac{1}{n}\sum_{k=1}^{n}(y_k - \bar{y}_k)$$

Where n is the number of data points, y_k are the actuals and $\bar{y}_k$ are the predicted values.

- *Accuracy:*

It answers the question: "How frequently does the model make correct predictions?". It can be calculated using this formula:

$$\mathrm{Acc} = \frac{\text{correct predictions}}{\text{total prediction}} = \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN}}$$

4.2.2. Execution environment and computing language

Executive environments have a crucial impact on the implementation and assessment of machine learning algorithms. They are the fundaments of an optimal algorithm infrastructure, considering compute/memory/data management needs. Through this work, we selected run times aimed at providing the flexibility & computers needed to power complex models and robustly process plenty of data. In the below we have used the environment and platforms:

- o Jupyter Notebook: It offers an interactive interface to execute cell-by-cell code with immediate results, supports more than 40 programming languages, and allows the integration of text formatted with Markdown or LaTeX for complex documents.
- o Roboflow: Allows users to easily upload, organize, and annotate image data sets while providing advanced data enhancement techniques to improve model robustness.
- o Kaggle: Allow running scripts or notebooks with 16 GB of RAM and GPU/TPU environments.

It seems crucial to include the computing language, and libraries, for the programming language we use Python programming language in general, and in detail, we use some built-in Python libraries, such as:

- o OpenCV: an open-source library specialized in computer vision tasks, it provides algorithms for object detection, facial recognition, and image processing, cv2 is the OpenCV implementation in Python.
- o Matplotlib: library dedicated to 2D data visualization, mainly used to create flexible charts.
- o NumPy: (Numerical Python) is a fundamental library for scientific computing in Python, a vast collection of mathematical functions for efficient operations.
- o Seaborn: is a data visualization library based on Matplotlib. It provides a high–level interface for drawing attractive and informative statistical graphs.
- o Scikit learn is an open-source library dedicated to Machine learning in Python, it provides efficient tools for predictive modeling (classification, regression, clustering, and dimension reduction).
- o TensorFlow: It's an open-source library created by Google, designed to make numerical computation and machine learning easier. Whether you're building or training large-scale deep learning models, this library has the tools to help you do it efficiently.

4.3. Results

After the training phase, we went to an evaluation to measure the efficacy of our OD–CNN–18 Layers. This step involved the calculation of several performance measures, such as accuracy, recall, F1_score, accuracy, and mAP, as well as other relevant metrics. The quality of our model was assessed. Table 4 shows the average performance measures obtained by our proposed approach.

The results shown in Table 4 display the average performance measures obtained by our OD–CNN–18 Layers approach. Precision at 96.77% indicates a high ability to avoid false positives, while recall at 99.47% highlights excellent detection of relevant objects. The F1-score of 98.07% reflects a solid balance between precision and recall, and the accuracy of 99.50% shows an impressive overall efficiency. The IoU (Intersection over Union) metric at 77.76% indicates a good match in the location of obstacles. The mAP (mean Average Precision) of 97.58% at IoU of 0.5 shows an accurate and consistent detection of obstacles, while the mAP@0.5:0.95, at 66.22%, demonstrates a robust performance on various IoU thresholds. Finally, a very low mean square error (MSE) of 0.016 indicates the accuracy of the enclosing box coordinates. In sum, the results show that the OD–CNN–18Layers excel at detecting and classifying obstacles while maintaining high accuracy under various conditions with a remarkable ability to avoid errors.

Table 5 shows the performance measurement by class for the OD–CNN–18 Layers model, providing a detailed view of its effectiveness in detecting pedestrians, animals, and vehicles. For the Animal class, an accuracy of 92.53% and recall of 99.34% gives an F1-score of 96.28%. The IoU of 67.72% and mAP@0.5 of 98.57% show good performance in animal detection, with a mAP@0.5:0.95 of 73.36% reflecting solid detection across various IoU thresholds. Finally, for the Vehicle class, the model reached an accuracy of 98.95% and a recall of 99.70%,

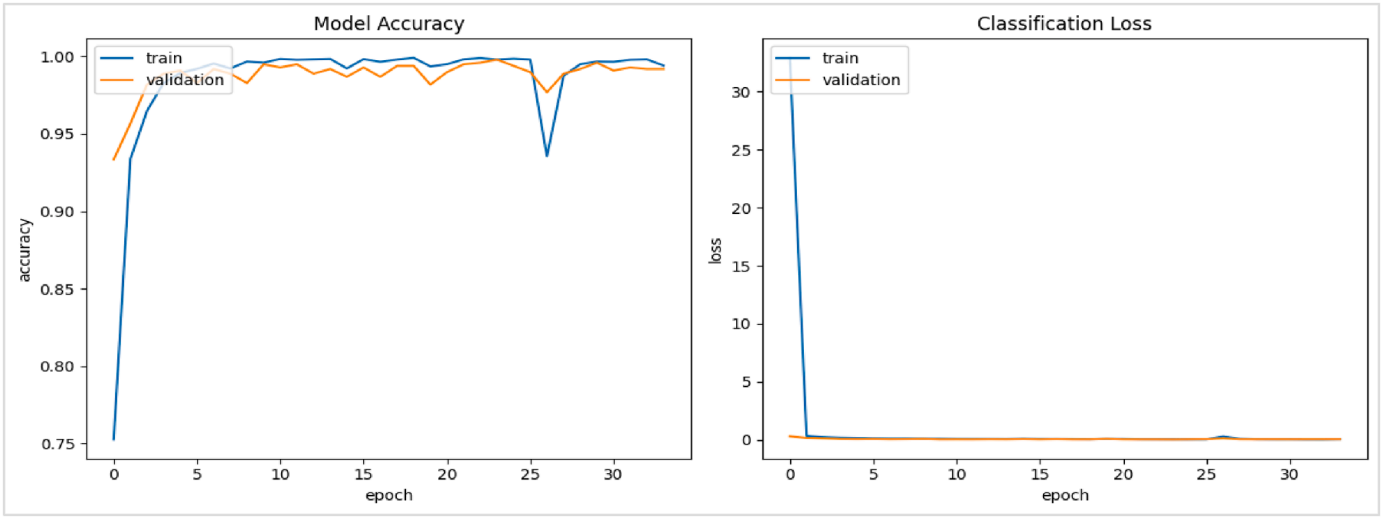


Fig. 12. Training and validation OD–CNN–18-layers accuracy and loss history.

resulting in an F1-score of 99.32%. The IoU of 80.52% and the mAP@0.5 of 97.00% highlight a robust vehicle detection, with a mAP@0.5:0.95 of 58.66%, indicating solid performance. In sum, the OD–CNN–18Layers model shows a remarkable overall performance, with particularly strong results for pedestrians and animals, and very good vehicle detection, this demonstrates its effectiveness in detecting and classifying road obstacles. The following figure provides a graphical representation of the results obtained by our OD–CNN–18Layers approach, thus allowing clear and comparative visualization of performance by class.

The suggested model was trained in over 100 epochs with a batch size 32 and a LR of 0.001. As shown in Fig. 12, the training results indicate that the accuracy varies between 0.93 and 0.99 starting from the second epoch for both the train and validation subsets. Additionally, it is important to highlight that the loss consistently trends towards zero from the third epoch onward, indicating improved model performance during the training process. The decrease in loss represents a successful reduction in prediction error, which indicates that the model is doing well at returning predictions closer to actual outcomes. This stable accuracy and decreasing loss show that the model is learning from the train data, but this is only of use if it generalizes (as predicted) unseen data.

Fig. 13 represents the confusion matrix, or error matrix in terms of the classifier algorithm, is a table that represents the number of instances in actual and predicted class instances. One of the evaluation metrics while evaluating a classification model is Confusion matrices. In object detection tasks, the bounding box loss curve is a graphical representation that tracks the loss values associated with the predictions of bounding rectangles during the learning phase of an object detection model across multiple epochs.

The model demonstrated strong performance by correctly identifying 655 pedestrians, 655 animals, and 664 vehicles. It misclassified 4 animals as pedestrians, 1 pedestrian as an animal, and 2 vehicles as animals, with no vehicles being incorrectly identified. Additionally, it missed no pedestrians but misclassified 3 animals as vehicles and 2 vehicles as animals. Overall, the model excels particularly with pedestrians and vehicles, as evidenced by high true positive counts and low false positives. While there are minor challenges in distinguishing between animals and vehicles, these result in only occasional misclassifications.

The bounding box loss for detecting dynamic obstacles, including pedestrians, vehicles, and animals, in Fig. 14 shows a consistent decline after the second epoch, trending towards zero as training progresses. This loss reduction continues steadily until the final epoch, indicating that the model is effectively refining its predictions and improving its accuracy in localizing these dynamic objects.

In addition to the confusion matrix, Fig. 15 illustrates the curve of ROC, displaying the AUC (area under the curve) for each class. The larger the area between the curves and the line $y = x$, the better the model's performance. The AUC values are 0.98 for pedestrians, 1.00 for

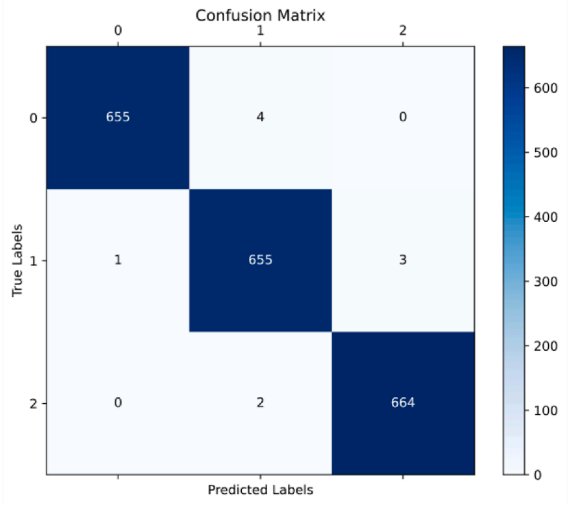


Fig. 13. D–CNN–18-layers confusion matrix.

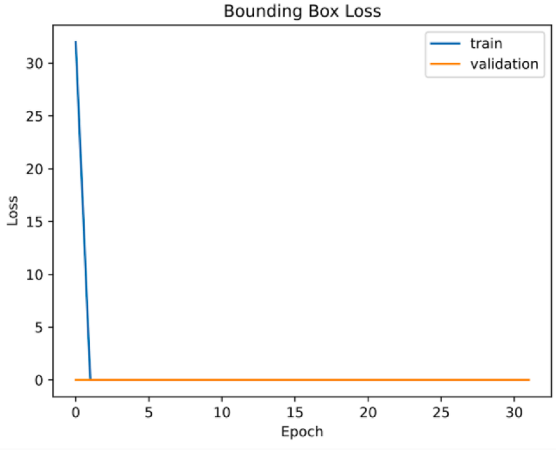


Fig. 14. Training and validation bounding box loss.

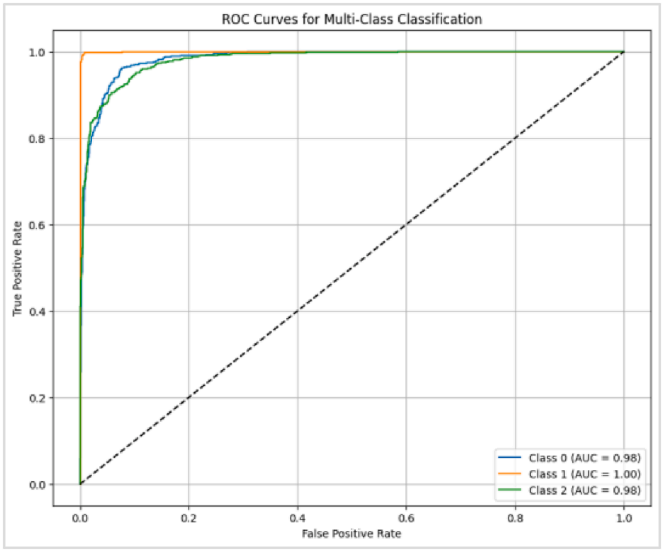


Fig. 15. Obstacle Detection–CNN–18-layers ROC curves.

animals, and 0.98 for vehicles. An AUC of 1.00 indicates perfect classification for animals, while the high values for pedestrians and vehicles suggest the model performs very well in distinguishing these classes, indicating overall strong predictive capability.

4.4. Our model evaluation compared to transfer learning models

In this section, we will compare our CNN model classifier and detector with transfer learning-based models, specifically YOLOv7, YOLOv8, and YOLOv9. We re-trained the YOLO models and fine-tuned them on our combined dataset, following the exact same pipeline as the OD–CNN–18 Layers model, which means same data preparation & preprocessing steps. The only difference lies in the architecture of each model.

To evaluate these pre-trained YOLO models, which we developed using the PVA dataset, we measured their performance against the same metrics as our OD–CNN–18 Layers approach, including accuracy, recall, F1-score, and mAP. All YOLO versions were trained on the same dataset and with identical hyperparameters, enabling a precise comparison with our proposed architecture. This assessment directly compares these established models and our novel approach.

After training our proposed model, OD–CNN–18 Layers, as well as the pre-trained models YOLOv7, YOLOv8, and YOLOv9, and evaluating their respective performance, we made a detailed comparison of the results obtained. The objective of this comparison is to highlight the weaknesses and strengths of each model in the domain of road obstacle detection and classification. Table 7 provides a comparison of our approach with pre-trained models, showing average performance measures and class results for each model.

As shown in Table 7, we presented the overall achievements of our proposed architecture with that of transfer learning models, including YoloV7, YoloV8, and YoloV9, all trained with the same hyperparameters as OD–CNN–18 Layers. The results indicate that all models performed well, but OD–CNN–18Layers truly stand out with an impressive accuracy and recall of 99.5%, along with an F1 score of 98.4%, highlighting its strong classification capabilities. However, its mAP@0.5:0.95 of 66.2% indicates some limitations in broader detection scenarios. On the other hand, YoloV8 shines with a higher mAP@0.5:0.95 score of 81.5%, making it particularly effective in complex detection tasks while maintaining solid precision (91.2%) and recall (92.5%). YoloV9 closely follows, boasting high precision (94.2%) but slightly lower recall (90.5%), which helps reduce false positives. Meanwhile, YoloV7 offers decent performance but doesn't match the metrics of the other models. Overall, if the goal is high classification accuracy, OD–CNN–18Layers is the top choice, whereas YoloV8 is better suited for robust detection across different thresholds.

The results of our OD–CNN–18 Layers model show excellent overall performance for pedestrian, animal, and vehicle detection. For pedestrians, the model attains an accuracy of 99.85% and a recall of 99.40%, which translates into a value of 99.62% of the f1 score. These values indicate that the model correctly identifies almost all pedestrians in the images with a very low proportion of false detections. The high Intersection over Union (IoU) rate of 85.22% and a mAP@0.5 of 97.15% reinforce this observation, demonstrating that the model can accurately delineate detected pedestrians while maintaining a mAP@0.5:0.95 of 66.65% which illustrates robustness across different IoU thresholds. For animals, the accuracy of 92.53%, high recall of 99.34%, and F1-score of 96.28% specify that the model is extremely accurate in detecting all animals present. With an IoU of 67.72% and a mAP@0.5 of 98.57%, these results show that the model achieves consistent and accurate animal detection, and the mAP@0.5:0.95 of 73.36% reflects solid performance across various IoU thresholds. For vehicles, the model shows an accuracy of 98.95% and a recall of 99.70%, resulting in an F1-score of 99.32%, with an overall accuracy of 99.62%. These results demonstrate the ability of the model to detect vehicles with a high degree of precision. The IoU of 80.52% and mAP@0.5 of 97.00% indicate a good performance in terms of location, while the mAP@0.5:0.95 of 58.66% suggests an acceptable robustness for vehicle detections at various thresholds.

The sharp drop in mAP@0.5:0.95 for the OD–CNN–18 Layers model at higher IoU thresholds can be attributed to its optimization for lower IoU values, where the model performs exceptionally well in object detection and recall. This limitation arises because the model is focused on achieving high recall and localization at lower thresholds but struggles with fine-tuning boundary detection at higher IoU thresholds. Factors such as anchor box size mismatches and the resolution of the receptive field may contribute to this issue, as they are better suited for detecting the presence of objects rather than refining bounding box boundaries. Additionally, the loss function used in training may prioritize object classification over precise localization. This issue will be addressed in future work, where we plan to explore enhancing the localization loss function and improving multi-scale feature extraction to better handle higher IoU thresholds and enhance detection accuracy.

To further expand on the comparative analysis with transfer learning models, the key difference in the performance of our CNN model lies in the high accuracy and recall, while yolov8 and yoloV9 excel in border detection tasks, particularly with their high mAP@0.5:0.95 scores, OD–CNN–18 Layers is exceptional when the task requires very specific classification such as identifying different types of pedestrians, animals, and vehicles in the same image. The efficacy of OD–CNN–18Layers on the mentioned tasks can be traced back to its design, which aims to maximize recall while using a strategy that minimizes false negatives. On the other hand, due to the wider range of detection intended, the YOLO models are geared towards broader precision and generalization universalization rather than finer classification specifics. So, OD–CNN–18Layers are more suitable for the more precise detection of certain specified classes, whereas YOLOv8 generally performs better in more diverse situations where multiple types of detections are required. This makes OD–CNN–18Layers more competent in tasks that require high recall and precision, while the YOLO versions are good for detection with different thresholds and for various tasks.

4.5. Our model evaluation compared to the state-of-the-art model's performance

In this section, we will analyze and compare our results with leading models to confirm the effectiveness of our suggested architecture. Given that no existing research has addressed or combined the three classes

Table 6
Runtime performance and computational cost.

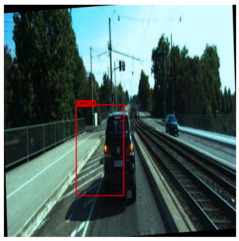
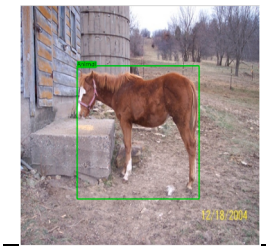
Obstacle detection	Class	Avg load time/s	Avg inference time/s	Avg save time/s	Avg CPU usage /%	GPU usage	Throughput / (pred·s^{-1})
	Vehicle	0.0010	0.8154	0.0160	10.6	No GPU found	1.2264 predictions/ second
	Pedestrian	0.0010	0.8714	0.0200	12.0	No GPU found	1.1476 predictions/ second
	Animal	0.0009	0.7420	0.0150	5.0	No GPU found	1.3477 predictions/ second

Table 7
Comparison of the results obtained.

Model	Accuracy	Precision	Recall	F1 Score	mAP@0.5	mAP@0.5:0.95
OD-CNN-18 Layers	**99.5 %**	**97.1 %**	**99.5 %**	**98.4 %**	**97.6 %**	66.2 %
YoloV8	93.6 %	91.2 %	92.5 %	91.8 %	96.3 %	**81.5 %**
YoloV9	93.3 %	94.2 %	90.5 %	92.3 %	95.2 %	80.0 %
YoloV7	94.3 %	91.6 %	90.8 %	91.2 %	94.7 %	76.1 %

Table 8
Model comparison with the state of art models on the KITTI dataset grouped by task.

Refs	Dataset	Goal	Model	Accuracy	Precision	Recall	F1-Score	mAP
[10]	KITTI	Pedestrian detection	R–CNN and RPN, skip pooling	–	–	–	–	76.30 %
[11]	KITTI	Pedestrian detection	Faster RCNN (ResNet 50)	–	–	–	–	58 %
Current	KITTI & Pascal VOc	Pedestrian detection	**OD-CNN-18-layer (ours)**	**99.40 %**	**99.90 %**	**99.40 %**	**99.60 %**	**97.20 %**
[21]	KITTI	Vehicle detection	MSCNN, MSCNN_PVHP	–	82.23 %	–	–	–
[22]	KITTI	Vehicle detection	RetinaNet	–	89.83 %	–	–	–
Current	KITTI & Pascal VOc	Vehicle detection	**OD-CNN-18-layer (ours)**	**99.60 %**	**99.00 %**	**99.70 %**	**99.30 %**	**97.00 %**
[19]	KITTI	Pedestrian and Vehicle detection	Faster R–CNN, Caffe framework, ZF net pre-trained	–	–	–	–	97.42 %
[18]	KITTI	Pedestrian and Vehicle detection	YOLOV4-5D	–	–	–	–	87.02 %

(pedestrian, vehicle, and animal), we will focus on class-level comparisons to ensure a credible assessment:

Table 8 compares different models for pedestrian and vehicle detection on the KITTI dataset. The OD–CNN–18-Layer (Ours) model outperforms others with exceptional accuracy (99.40%) and mAP (97.20%) for pedestrian detection, and accuracy (99.60%) and mAP (97.00%) for vehicle detection. Its precision (99.90%) and recall (99.70%) for vehicle detection highlight its strong capabilities. In comparison, older models like R–CNN and RPN, Skip pooling, and Faster RCNN (ResNet 50) show much lower mAP values of 76.30% and 58%, respectively. The MSCNN and RetinaNet models show good precision but lack complete performance metrics. Faster R–CNN (Caffe framework, ZF Net pre-trained) has a high mAP (97.42%) but lacks other metrics, while YOLOV4-5D performs decently with an mAP of 87.02%. In conclusion, OD–CNN–18-Layer is the top performer across all metrics, clearly outperforming other models.

Table 9 provides an overview of the results of our model compared to the state-of-the-art models. The OD–CNN–18-Layer (Ours) model

Table 9
model comparison based on the goal of the paper (Task).

Refs	Dataset	Dataset	Model	Accuracy	Precision	Recall	F1-Score	mAP
[17]	Animal detection	COCO	YOLOv3	–	–	–	–	78.00 %
[25]	Animal detection	Custom dataset	RetinaNet, YOLOv3, faster RCNN	–	–	–	–	–
[28]	Animal detection	Animals dataset	HOG + cascading classifier	82.50 %	–	–	–	–
Current	Animal detection	**PVA dataset**	**OD-CNN-18-layer (ours)**	**99.40 %**	**92.50 %**	**99.30 %**	**96.30 %**	**98.60 %**
[23]	Obstacle detection	Roboflow, CVD	YOLOv8	62.90 %	45.30 %	–	–	47.50 %
[10]	Pedestrian detection	KITTI	R–CNN, RPN, skip pooling	–	–	–	–	76.30 %
[12]	Pedestrian detection	Pedestrian images	CNN, AdaBoost	–	–	–	–	–
[13]	Pedestrian detection	ZUT	YOLOv3	–	–	–	–	89.10 %
[15]	Pedestrian detection	COCO (training)	Faster-RCNN	39.40 %	–	–	–	–
[16]	Pedestrian detection	VOC2014	YOLOv3	–	74.38 %	–	–	–
[24]	Pedestrian detection	Captured images of pedestrians	YOLOv5	75.00 %	–	–	–	–
[27]	Pedestrian detection	CIFAR-10	DCGAN + SSD	–	–	–	–	–
[29]	Pedestrian detection	INRIA, PSU CVC05, WU	CNN,	65.20 %	–	–	–	44.00 %
[29]	Pedestrian detection	INRIA, PSU CVC05, WU	MobileNet	98.69 %	–	–	–	78.00 %
[31]	Pedestrian detection	Custom dataset	CNN (AlexNet)	72.00 %	–	–	–	–
[9]	Pedestrian detection	INRIA	HOG, LBP, XGBoost	–	97.00 %	97.00 %	97.00 %	–
Current	Pedestrian detection	**PVA dataset**	**OD-CNN-18-layer (ours)**	**99.40 %**	**99.90 %**	**99.40 %**	**99.60 %**	**97.20 %**
[21]	Vehicle detection	KITTI	MSCNN, MSCNN_PVHP	–	82.23 %	–	–	–
[22]	Vehicle detection	KITTI	RetinaNet	–	89.83 %	–	–	–
[30]	Vehicle detection	BDD + Udacity	CNN + SVM	98.69 %	–	–	–	–
Current	Vehicle detection	**PVA dataset**	**OD-CNN-18-layer (ours)**	**99.60 %**	**99.00 %**	**99.70 %**	**99.30 %**	**97.00 %**

outperforms all other models in both pedestrian and vehicle detection across various datasets, achieving exceptional results with high accuracy, precision, and recall, F1-score, and mAP. It surpasses models like YOLOv8, Faster RCNN, and RetinaNet, which, although strong, fall short in comparison to OD–CNN–18-Layer's comprehensive detection capabilities. Notably, the OD–CNN–18-Layer model shows remarkable performance on the custom dataset, consistently exceeding 99% in key metrics, making it the most robust and effective model in this comparison.

By comparing these results with those of the state-of-the-art models, our OD–CNN–18 Layers model demonstrates a remarkable superiority in several aspects. For pedestrian detection, for example, reference models like RCNN [10], Faster RCNN [11] and YoloV4-TD [18] have achieved a mAP of 76.30%, 58.00%, and 87.02% respectively on the KITTI dataset, in comparison to our solution which has an overall mAP of 97.6%, and an mAP of 97.2% for only pedestrian detection. Under the same task which is pedestrian detection using other datasets like VOC2014 in Ref. [16] and COCO in Ref. [17] the models YoloV3 and Faster RCNN achieved a mAP of 74.38% and 39.4% respectively on these datasets, but these results are still far below our OD–CNN–18Layers performance. For animal detection, state-of-the-art the model YoloV3 [17] achieves 78.00% mAP on a combination of COCO. Our model shows a mAP@0.5 of 98.6%, thus far surpassing these results. Regarding vehicle detection, several models tested on the KITTI dataset [11] such as MobileNetV2 SSD (AP = 27.9%), InceptionV2 SSD (AP = 27.5%) and other MSCNN-based architectures show up to 82.23% APs [21]. Our model, on the other hand, has an mAP@0.5 of 97.00% showing more robust and accurate performance. In addition, a study using YOLOv8 for obstacle detection while driving [23] reported an accuracy of 62.90%, a recall of 45.30%, and a mAP of 47.50%. Our model, on the other hand, has significantly higher performance with an accuracy of 99.46%, a recall of 99.47%, and a mAP@0.5 reaching up to 97.6%. In sum, our OD–CNN–18Layers model outperforms existing state-of-the-art approaches, offering significant improvements to accuracy, recall, and overall robustness, while demonstrating its ability to generalize effectively over different classes of obstacles. This confirms its relevance for practical application in complex road environments.

Finally, before wrapping up this paper it is necessary to touch upon the possibilities that our proposal brings for further developing autonomous vehicles and reducing fatalities. Enhancing the recognition and categorization of pedestrians, vehicles, and animals will reduce the probability of accidents while increasing the dependability of self-driving systems by enforcing our architecture. This, in turn, would reduce accident costs by a large margin, delivering great socio-economic benefits. Additionally, we may be helping to create a more enabling environment for autonomous technologies which could enable the realization of an increased safety and mobility future.

4.6. Ethical implications and potential limitations in real-world deployment

In practical scenarios, especially in fields like autonomous driving and surveillance, the effectiveness of the OD–CNN–18 Layers model is significantly affected by its rates of false positives and false negatives. False positives occur when the model mistakenly detects an obstacle that is not present, while false negatives refer to instances where the model fails to recognize an actual obstacle. The evaluation results show that the model demonstrates strong performance in identifying barriers, with high accuracy and recall rates, indicating a minimal occurrence of false negatives. However, the precision rate of 97.1% reveals that about 2.9% of the detections are false positives. Although this percentage may seem low, in safety-sensitive environments like autonomous driving, even a few false positives can result in unnecessary actions, potentially leading to traffic disruptions or accidents. In conclusion, while the OD–CNN–18Layers model shows great promise, it is essential to carefully consider the ethical implications associated with false positives and false negatives. Addressing these issues will be a priority to improve the model's performance and ensure its responsible use in real-world applications.

4.7. Runtime performance and computational cost

Assessing a model's computational efficiency is crucial for determining its practicality, especially in environments with limited resources or time constraints. In this section, we thoroughly evaluate the computational performance of our CNN model since it's the one with best results, by analyzing metrics such as image loading time, inference time, output saving time, CPU and GPU utilization, and inference throughput. These metrics are measured across each class in the dataset to ensure a detailed and comprehensive assessment. This analysis sheds light on the model's overall efficiency, highlights performance variations between classes, and identifies opportunities for optimization in real-world scenarios. While the model was trained using Kaggle's GPU resources, all testing was conducted locally on a CPU-only system without access to GPU acceleration.

Table 6 provides insights into the computational performance of our obstacle detection model for three classes: Vehicle, Pedestrian, and Animal. It includes metrics for image load time, inference time, save time, CPU usage, GPU availability, and inference throughput. These metrics reflect the model's efficiency and potential computational bottlenecks during the testing phase.

- Inference time variability: detecting pedestrians takes the longest inference time, which could be due to more complex features or a higher level of variability in pedestrian images.
- CPU usage trends: CPU utilization aligns with the inference time, with pedestrians requiring the most computational resources and animals the least.
- Throughput comparison: The model achieves the highest throughput for animals, indicating faster processing, while pedestrians result in the lowest throughput due to longer inference times.
- Efficiency across classes: Despite the absence of a GPU, the model demonstrates consistent load times and reasonable inference times. However, optimizing the pedestrian detection process could improve overall performance

5. Conclusion

This research has advanced the detection and classification of road obstacles by implementing CNN. The detection of mobile obstacles enabled us to assess the efficiency of our OD–CNN–18 Layers model in various contexts and compare their results with existing methods. The experiment focused on detecting dynamic obstacles, including pedestrians, animals, and vehicles, using the OD–CNN–18Layers model. The results showed that this model offers a remarkable performance for each obstacle category. Compared to pre-existing models, our approach has shown superiority, highlighting its effectiveness in detecting and classifying mobile obstacles under various conditions. This demonstrates the ability of our model to function efficiently and reliably in dynamic and complex situations.

The results of our experiments show that our approach to mobile barriers makes significant contributions to the advancement of obstacle detection. The OD–CNN–18 Layers offers innovative and powerful solutions that surpass those available on the market. This model shows significant potential for integration into ITS, aiming to improve not only road safety but also infrastructure efficiency, the decision-making in various fields like autonomous vehicles, traffic management and safety systems, the ability to quickly and accurately detect and classify road obstacles will contribute to safer driving conditions, reduce response times in critical situations, and enhance overall system performance. Furthermore, the model's adaptability to diverse environments and obstacle types makes it a valuable tool for deployment in both urban and rural settings, where road conditions and challenges vary; However, our CNN model has limitations, especially in multi-object detection and classification. While it struggles to detect and classify multiple objects within the same frame, our implementation of transfer models—YOLOv7, YOLOv8, and YOLOv9—effectively addresses this challenge. Nonetheless, it's worth noting that the detection and classification metrics for these models remain lower than those of our OD–CNN–18Layers model. The generalizability of the dataset is another point that needs to be taken into consideration, because even though our custom dataset offers valuable resources for training and evaluating object detection models. However, the primary focus on specific environments and scenarios which may limit the generalizability of models trained exclusively on them to unseen or diverse real-world conditions, the KITTI dataset provides a rich set of examples for these scenarios, it may not encompass other environments like indoor settings, varied weather conditions, or different geographical regions. Similarly, the PASCAL VOC 2012 dataset includes images of animals, vehicles, and other objects in various indoor and outdoor settings. Despite its diversity, it may not cover all possible real-world scenarios, such as specific animal species in certain habitats or objects in unique contexts. Despite its variety, it does not comprehensively cover all real-world scenarios, such as specific animal species in unique habitats or objects in uncommon contexts—traffic sign detection or fallen trees in streets, for instance. These scenarios will be addressed in a future study focusing on fixed obstacles (non-dynamic obstacles) such as potholes, cracks, traffic signs, and other static challenges.

The prospects for future research and development are particularly promising. An interesting avenue would be to explore the adaptability of our models to other types of obstacles or to various road conditions that were not covered in this study. In addition, the integration of our models into real-time detection systems for autonomous vehicles or road monitoring systems could significantly improve real-time obstacle management. Optimizing the model's performance for execution on embedded platforms or low-power devices is also a crucial aspect that could facilitate wider adoption in practical applications. The enrichment of data by complementary sources such as images captured by drones or LIDAR sensors represents another opportunity for improvement. These technologies could provide additional information to increase the accuracy of detection in a variety of contexts. Although our model has already shown remarkable performance, the field of road obstacle detection is constantly evolving. The results open the way for innovations and allow for significant future advances in the efficiency and safety of ITS.

In sum, this research has laid the foundation for a better understanding and application of road obstacle detection. Our contribution opens avenues for future developments and offers perspectives for greater integration of advanced technologies in road infrastructure management and overall road safety.

CRediT authorship contribution statement

Hamza Assemlali: Writing – review & editing. **Soukaina Bouhsissin:** Writing – review & editing, Conceptualization. **Nawal Sael:** Writing – review & editing.

Data availability

The data and materials used to support the findings of this study are available from the corresponding author upon reasonable request.

Declaration of competing interest

The authors declare that there are no conflicts of interest related to this manuscript titled: “Deep Learning-Driven CNN Model for detection and classification of Moving Obstacles.”

All authors have disclosed any financial or personal relationships that could influence the research presented in this study. Furthermore, there are no financial sponsors or funding sources that may have influenced the content of this manuscript.

If any potential conflicts arise in the future, the authors commit to disclose them promptly.

Acknowledgements

No funding.

References

[1] Lin Y, Wang P, Ma M. Intelligent transportation system(ITS): concept, challenge and opportunity. In: 2017 IEEE 3rd international conference on big data security on cloud (BigDataSecurity), IEEE international conference on high performance and smart computing, (HPSC) and IEEE international conference on intelligent data and security (IDS). IEEE; May 2017. p. 167–72. https://doi.org/10.1109/BigDataSecurity.2017.50.

[2] Escorcia Guzman JH, Zuluaga-Ortiz RA, Barrios-Miranda DA, Delahoz-Dominguez EJ. Information and communication technologies (ICT) in the processes of distribution and use of knowledge in higher education institutions (HEIs).

Procedia Comput Sci Jan. 2022;198:644–9. https://doi.org/10.1016/J.PROCS.2021.12.300.
[3] Wozniak D, Shahini F, Nasr V, Zahabi M. Analysis of advanced driver assistance systems in police vehicles: a survey study. Transport Res F Traffic Psychol Behav Nov. 2021;83:1–11. https://doi.org/10.1016/J.TRF.2021.09.017.
[4] Lodhi SS, Kumar N, Pandey PK. Autonomous vehicular overtaking maneuver: a survey and taxonomy. Vehicular Communications Aug. 2023;42:100623. https://doi.org/10.1016/J.VEHCOM.2023.100623.
[5] Pedestrian detection | papers with code." Accessed: September. 29, 2024. [Online]. Available: https://paperswithcode.com/task/pedestrian-detection.
[6] Mouha RA, Mouha RA. Internet of Things (IoT). J Data Anal Inf Process Mar. 2021;9(2):77–101. https://doi.org/10.4236/JDAIP.2021.92006.
[7] Geiger A, Lenz P, Urtasun R. Are we ready for autonomous driving? the KITTI vision benchmark suite. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition; 2012. p. 3354–61. https://doi.org/10.1109/CVPR.2012.6248074.
[8] VOC - ultralytics YOLO docs." Accessed: September. 23, 2024. [Online]. Available: https://docs.ultralytics.com/datasets/detect/voc/.
[9] Jiang Y, Tong G, Yin H, Xiong N. A pedestrian detection method based on genetic algorithm for optimize XGBoost training parameters. IEEE Access 2019;7:118310–21. https://doi.org/10.1109/ACCESS.2019.2936454.
[10] Liu J, Gao X, Bao N, Tang J, Wu G. Deep convolutional neural networks for pedestrian detection with skip pooling," Proceedings of the International Joint Conference on Neural Networks. Jun. 2017. p. 2056–63. https://doi.org/10.1109/IJCNN.2017.7966103. 2017-May.
[11] Chen L, Lin S, Lu X, Cao D, Wu H, Guo C, et al. Deep neural network based vehicle and pedestrian detection for autonomous driving: a survey. IEEE Trans Intell Transport Syst Jun 2021;22(6):3234–46. https://doi.org/10.1109/TITS.2020.2993926.
[12] Chen Z, Huang X. Pedestrian detection for autonomous vehicle using multi-spectral cameras. IEEE Transactions on Intelligent Vehicles Jun. 2019;4(2):211–9. https://doi.org/10.1109/TIV.2019.2904389.
[13] Tumas P, Nowosielski A, Serackis A. Pedestrian detection in severe weather conditions. IEEE Access 2020;8:62775–84. https://doi.org/10.1109/ACCESS.2020.2982539.
[14] Barba-Guaman L, Naranjo JE, Ortiz A. Deep learning framework for vehicle and pedestrian detection in rural roads on an embedded GPU. Electronics Mar. 2020;9(4):589. https://doi.org/10.3390/ELECTRONICS9040589. 2020, Vol. 9, Page 589.
[15] Razzok M, Badri A, EL Mourabit I, Ruichek Y, Sahel A. Pedestrian detection system based on deep learning. Int J Adv Appl Sci Sep. 2022;11(3):194–8. https://doi.org/10.11591/IJAAS.V11.I3.PP194-198.
[16] Razzok M, Badri A, Mourabit IEL, Ruichek Y, Sahel A. Pedestrian detection under weather conditions using conditional generative adversarial network. IAES Int J Artif Intell Dec. 2023;12(4):1557–68. https://doi.org/10.11591/ijai.v12.i4.pp1557-1568.
[17] Yudin D, Sotnikov A, Krishtopik A. Detection of big animals on images with road scenes using deep learning. In: Proceedings - 2019 international conference on artificial intelligence: applications and innovations, IC-aiai 2019; Sep. 2019. p. 100–3. https://doi.org/10.1109/IC-AIAI48757.2019.00028.
[18] Cai Y, Luan T, Gao H, Wang H, Chen L, Li Y, et al. YOLOv4-5D: an effective and efficient object detector for autonomous driving. IEEE Trans Instrum Meas 2021;70. https://doi.org/10.1109/TIM.2021.3065438.
[19] Prabhakar G, Kailath B, Natarajan S, Kumar R. Obstacle detection and classification using deep learning for tracking in high-speed autonomous driving. In: TENSYMP 2017 - IEEE international symposium on technologies for smart cities; Oct. 2017. https://doi.org/10.1109/TENCONSPRING.2017.8069972.
[20] Qi G, Wang H, Haner M, Weng C, Chen S, Zhu Z. Convolutional neural network based detection and judgement of environmental obstacle in vehicle operation. CAAI Trans Intell Technol Jun. 2019;4(2):80–91. https://doi.org/10.1049/TRIT.2018.1045.
[21] Kim J, Hong S, Kim E. Novel on-road vehicle detection system using multi-stage convolutional neural network. IEEE Access 2021;9:94371–85. https://doi.org/10.1109/ACCESS.2021.3093698.
[22] Wang H, Yu Y, Cai Y, Chen X, Chen L, Liu Q. A comparative study of state-of-the-art deep learning algorithms for vehicle detection. IEEE Intelligent Transportation Systems Magazine Jun. 2019;11(2):82–95. https://doi.org/10.1109/MITS.2019.2903518.
[23] Sharma T, Chehri A, Fofana I, Jadhav S, Khare S, Debaque B, et al. Deep learning-based object detection and classification for autonomous vehicles in different weather scenarios of Quebec, Canada. IEEE Access 2024;12:13648–62. https://doi.org/10.1109/ACCESS.2024.3354076.
[24] Malbog MA, Marasigan R, Mindoro J, Mortos YC, Ilustre LF. PED-AI: pedestrian detection for autonomous vehicles using YOLOv5," E3S web of conferences, vol. 488; Feb. 2024. p. 3013. https://doi.org/10.1051/E3SCONF/202448803013.
[25] Singh A, Pietrasik M, Natha G, Ghouaiel N, Brizel K, Ray N. Animal detection in man-made environments. In: Proceedings - 2020 IEEE winter conference on applications of computer vision. WACV 2020; Oct. 2019. p. 1427–38. https://doi.org/10.1109/WACV45572.2020.9093504.
[26] Sadik MN, Hossain T, Sayeed F. Real-time detection and analysis of vehicles and pedestrians using deep learning. Apr. 2024 [Online]. Available: https://arxiv.org/abs/2404.08081v1. [Accessed 22 September 2024].
[27] Dinakaran RK, Easom P, Bouridane A, Zhang L, Jiang R, Mehboub F, et al. Deep learning based pedestrian detection at distance in smart cities. Adv Intell Syst Comput 2020;1038:588–93. https://doi.org/10.1007/978-3-030-29513-4_43.
[28] Sharma SU, Shah DJ. A practical animal detection and collision avoidance system using computer vision technique. IEEE Access 2017;5:347–58. https://doi.org/10.1109/ACCESS.2016.2642981.
[29] Thu M, Suvonvorn N, Kittiphattanabawon N. Pedestrian classification on transfer learning based deep convolutional neural network for partial occlusion handling. Int J Electr Comput Eng 2023;13(3):2812–26. https://doi.org/10.11591/ijece.v13i3.pp2812-2826.
[30] Hu J, Sun Y, Xiong S. Research on the cascade vehicle detection method based on CNN. Electronics Feb. 2021;10(4):481. https://doi.org/10.3390/ELECTRONICS10040481. 2021, Vol. 10, Page 481.
[31] Gao F, Wang C, Li C. A combined object detection method with application to pedestrian detection. IEEE Access 2020;8:194457–65. https://doi.org/10.1109/ACCESS.2020.3031005.
[32] COCO - common objects in context." Accessed: September. 23, 2024. [Online]. Available: https://cocodataset.org/#home.
[33] Li M, Zhang L, Li L, Song W. Yolo-based traffic sign recognition algorithm. Comput Intell Neurosci 2022;2022. https://doi.org/10.1155/2022/2682921.
[34] Kim Y, Park K, Kim M, Kum D, Choi JW. 3D dual-fusion: dual-domain dual-query camera-LiDAR fusion for 3D object detection. Nov. 2022 [Online]. Available: http://arxiv.org/abs/2211.13529. [Accessed 23 September 2024].